

PowerPredict : ML Based Bill Prediction System

A Minor Project Report

Submitted in partial fulfillment of requirement of the
Degree of

**BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE &
ENGINEERING**

BY

Minhaj Ahmed Sheikh (EN23CS301616)

**Mohammed Furqan Shaikh
(EN23CS301626)**

Mo. Fazil Ahmed Ghori (EN23CS301618)

Under the Guidance of

Prof. Dr. Sheetal Bawane

Prof. Dr. Garima Silakari Tukra



Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE- 453331

APRIL-2026

Report Approval

The project work "**PowerPredict : ML Based Bill Prediction System**" is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn there in; but approve the "Project Report" only for the purpose for which it has been submitted.

Internal Examiner 1

Prof. Dr Sheetal Bawane

Assistant Professor

Medicaps University

Internal Examiner 2

Prof. Dr Garima Silakari Tukra

Assistant Professor

Medicaps University

External Examiner

Name:

Designation

Affiliation

Declaration

I/We hereby declare that the project entitled " **PowerPredict : ML Based Bill Prediction System** " submitted in partial fulfillment for the award of the degree of Bachelor of Technology in 'Computer Science & Engineering' completed under the supervision of **Prof. Dr. Sheetal Bawane** and **Prof. Dr. Garima Silakari Tukra**, Faculty of Engineering, Medi-Caps University Indore is an authentic work.

Further, I/we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature and name of the student(s) with date

_____Minhaj Ahmed Sheikh

_____Mohammed Furqan Shaikh

_____Mo. Fazil Ahmed Ghori

Certificate

I/We, Prof. **Dr. Sheetal Bawane** and Prof. **Dr. Garima S Tukra** certify that the project entitled **“PowerPredict : ML Based Bill Prediction System”** submitted in partial fulfillment for the award of the degree of Bachelor of Technology by **Minhaj Ahmed Sheikh, Mohammed Furqan Shaikh, Mo. Fazil Ahmed Ghor** is the record carried out by him/them under my/our guidance and that the work has not formed the basis of award of any other degree elsewhere.

Prof. Dr. Sheetal Bawane

Computer Science & Engineering

Medi-Caps University, Indore

Prof. Dr. Garima Silakari Tukra

Computer Science & Engineering

Medi-Caps University, Indore

Dr. Kailash Chandra Bandhu

Head of the Department

Computer Science & Engineering

Medi-Caps University, Indore

Acknowledgements

We would like to express our deepest gratitude to the Honorable Chancellor, **Shri R C Mittal**, who has provided us with every facility to successfully carry out this project, and our profound indebtedness to **Prof. (Dr.) D. K. Patnaik**, Vice Chancellor, Medi-Caps University, whose unfailing support and enthusiasm has always boosted our morale. We also thank Prof. **(Dr.) Ratnesh Litoriya**, Dean, Faculty of Engineering, Medi-Caps University, for giving us a chance to work on this project. We would also like to thank the Head of the Department **Dr. Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

It is their help and support, due to which we became able to complete the design and technical report. Without their support this report would not have been possible.

We would also like to express our heartfelt gratitude to our project guides, **Prof. Dr. Sheetal Bawane** and **Prof. Dr. Garima Silakari Tukra**, for their continuous guidance, expert advice, and valuable feedback throughout the development of " PowerPredict : ML Based Bill Prediction System ".

Minhaj Ahmed Sheikh

Mohammed Furqan Shaikh

Mo. Fazil Ahmed Ghori

B.Tech. III Year (K)

Department of Computer Science & Engineering

Faculty of Engineering

Medi-Caps University, Indore

Abstract

The rapid growth of digital technology has significantly transformed how individuals monitor and manage their electricity consumption. Traditional approaches, such as manually checking monthly bills or using basic tracking methods, primarily focus on past usage without offering predictive insights or actionable recommendations. As a result, users remain reactive rather than proactive, often leading to inefficient energy consumption and unexpectedly high electricity costs.

"PowerPredict – ML Based Electricity Bill Prediction System" is a full-stack web application that integrates data input, analysis, prediction, and visualization into a unified platform. The system enables users to estimate their monthly electricity bill based on appliance usage patterns, including usage hours and appliance counts. Key features include secure JWT-based authentication, an interactive dashboard for entering appliance data, real-time electricity bill prediction, appliance-level energy consumption breakdown, and persistent storage of user prediction history using a relational database.

The system follows a modular full-stack architecture. The frontend is developed using React.js, providing a responsive and intuitive user interface. The backend is implemented using FastAPI, ensuring high-performance REST API communication and efficient request handling. SQLite is used as the database for storing user data and prediction history, maintaining lightweight and reliable data persistence. A machine learning model based on the Random Forest algorithm is integrated into the backend, enabling accurate prediction of electricity consumption and billing based on engineered features.

The defining feature of this system is its practical predictive intelligence, where users receive immediate, data-driven estimates of their electricity expenses along with insights into consumption patterns. This helps users make informed decisions to optimize energy usage and reduce costs. The project demonstrates the effective integration of full-stack web development, machine learning, and data-driven decision support applied to the domain of electricity consumption management.

Keywords: React.js, FastAPI, SQLite, Random Forest, Electricity Bill Prediction, Machine Learning, RESTful API, JWT Authentication, Energy Consumption, Predictive Modeling, Data Visualization, Smart Energy Management.

Table of Contents

Section	Title	Page No.
Report Approval		ii
Declaration		iii
Certificate		iv
Acknowledgement		v
Abstract		vi
Table of Contents		vii
Abbreviations		x
Notations & Symbols		xi
Chapter 1	Introduction	1
1.1	Introduction	1
1.2	Literature Review	1
1.3	Objectives	2
1.4	Significance	3
Chapter 2	Requirements Specification	5
2.1	User Characteristics	5
2.2	Functional Requirements	6
2.3	Software Requirements	7
Chapter 3	System Design	8
3.1	System Architecture	9
3.2	Data Flow Diagrams (Level 0, Level 1)	10
3.3	Activity Diagram	10
3.4	Class Diagram	11
3.5	ER Diagram	11
3.6	Database Design	12
Chapter 4	Implementation, Testing and Maintenance	13
4.1	Languages, IDEs, Tools and Technologies	13
4.2	Testing Techniques and Test Plans	14
4.3	Installation Instructions	15
4.4	End User Instructions	16
Chapter 5	Results and Discussion	16
5.1	User Interface Representation	17

5.2	Description of System Modules	18
5.3	Snapshots of System	19
5.4	Backend Representation	21
5.5	Database Representation	22
Chapter 6	Summary and Conclusions	23
Chapter 7	Future Scope	23
	Appendix	24
	Bibliography	25

Chapter-1

Introduction

1.1 Introduction

In the modern digital era, the integration of technology in monitoring and managing electricity consumption has become increasingly essential. Despite the availability of electricity billing systems and basic tracking methods, most existing approaches focus only on recording past usage without providing meaningful insights, predictive analysis, or proactive recommendations. As a result, users remain reactive rather than proactive in managing their electricity consumption, often realizing high bills only after the billing cycle is completed.

The proposed system, "**PowerPredict – AI-Based Electricity Bill Prediction System**", is a full-stack web application designed to transform traditional electricity monitoring into a proactive and intelligent decision-support system. The application enables users to input appliance usage details, including operating hours and appliance counts, to receive real-time predictions of their monthly electricity bill along with insights into consumption patterns.

The defining feature of this system is its **practical predictive intelligence**, where users receive immediate and data-driven estimates of their electricity bills along with an understanding of how different appliances contribute to overall consumption. This transforms the application into a powerful decision-support tool for efficient energy management rather than a simple tracking system.

1.2 Literature Review

Various electricity consumption estimation and monitoring systems have been developed to assist users in understanding their energy usage and managing electricity costs. Existing solutions such as traditional billing systems, smart meter dashboards, and basic energy tracking applications provide functionalities like recording past consumption, displaying unit usage, and generating monthly bills.

Recent research in energy analytics emphasizes the importance of integrating machine learning and data-driven approaches into electricity management systems. Modern solutions must focus on analyzing consumption patterns, predicting future electricity usage, and providing actionable insights for efficient energy utilization.

However, existing systems exhibit several critical limitations: they lack accurate prediction capabilities for future electricity bills, provide limited insight into the impact of individual appliance usage, generate only reactive information after consumption has occurred, offer minimal support for proactive decision-making, and have limited integration of intelligent analytics into user-friendly applications.

Compared to existing solutions, the proposed PowerPredict system offers real-time electricity bill prediction based on user-specific appliance usage patterns, integration of machine learning using the Random Forest algorithm for improved accuracy, an interactive and user-friendly interface for input and visualization, and persistent storage of prediction history for better tracking and analysis. The system aims to provide a smart, predictive, and user-centric approach to electricity consumption management.

1.3 Objectives

The primary objectives of the proposed system are:

- To develop a web-based **Electricity Bill Prediction System** using React.js, FastAPI, and SQLite
- To implement secure user authentication using JWT tokens and password hashing mechanisms
- To enable users to input detailed appliance usage data including operating hours and appliance counts
- To provide accurate electricity bill prediction using a machine learning model based on the Random Forest algorithm
- To implement real-time prediction processing through efficient REST API communication
- To analyze electricity consumption patterns based on user input data and identify usage trends
- To generate reliable estimates of electricity consumption and billing in real time

1.4 Significance of the Project

The significance of this project lies in its ability to transform passive electricity bill observation into active, intelligent energy management. The system reduces dependency on manual estimation and enhances the overall user experience through predictive intelligence and real-time analysis.

Key benefits include:

- Improved efficiency in electricity consumption management through automation and data-driven prediction
- Proactive estimation of electricity bills that helps users anticipate costs before the billing cycle ends
- Consumption insights that help users understand how appliance usage impacts overall electricity expenses
- Data-driven predictions that improve user awareness and support better energy usage decisions
- A user-friendly interface that enables easy input and instant visualization of results
- Secure and reliable authentication with structured data storage ensuring consistency and integrity

This project is particularly valuable as a demonstration of full-stack web development integrated with machine learning and data analytics, addressing a real-world problem of managing electricity consumption and reducing unexpected energy costs.

Chapter-2

Requirements Specification

2.1 User Characteristics

The system supports two user types:

1. Standard User

- Basic web usage knowledge
- Can register, login, enter appliance data, and view predictions
- No technical expertise required

2. System Administrator

- Manages system performance and database
- Monitors activity and updates models
- Has full system access

2.2 Functional Requirements

Functional requirements define what the system shall do. Each requirement is assigned a unique identifier for traceability.

User Authentication

- REQ-1: Users shall register with valid credentials
- REQ-2: System shall validate email and prevent duplicates
- REQ-3: Passwords shall be securely hashed
- REQ-4: JWT-based login shall be implemented
- REQ-5: Only authorized users shall access data

Electricity Usage Input

- REQ-6: Users shall input appliance hours and counts
- REQ-7: System shall validate inputs
- REQ-8: Users shall modify inputs before prediction
- REQ-9: Multiple appliances shall be supported

Prediction System

- REQ-10: System shall predict electricity bills
- REQ-11: Random Forest model shall be used
- REQ-12: Units and bill shall be estimated
- REQ-13: Results shall be shown in real time

Data Storage

- REQ-14: System shall store predictions
- REQ-15: Users shall view history
- REQ-16: Data shall be user-specific

2.3 Non-Functional Requirements

Performance

- NFR-1: System shall respond quickly
- NFR-2: Predictions shall be generated instantly

Security

- NFR-3: Passwords shall be hashed
- NFR-4: JWT authentication shall be used
- NFR-5: User data shall remain secure

Reliability

- NFR-6: System shall provide consistent results
- NFR-7: Errors shall be handled properly

Usability

- NFR-8: Interface shall be simple and user-friendly
- NFR-9: Predictions shall require minimal steps

Chapter 3

System Design

3.1 Introduction

System design defines the overall architecture, components, modules, interfaces, and data flow of the system. The "**PowerPredict – AI-Based Electricity Bill Prediction System**" follows a modular full-stack architecture to ensure separation of concerns, scalability, and efficient communication between components.

3.2 System Architecture

The system architecture consists of four main layers:

1. Presentation Layer (Frontend)

- React.js web application providing a responsive and interactive user interface
- Communicates with backend via RESTful APIs using Axios
- Displays input forms, prediction results, and user history

2. Application Layer (Backend)

- FastAPI framework handles API requests and business logic
- Implements authentication, input validation, and prediction processing
- Integrates the machine learning model for bill prediction
- Communicates with the database for storing and retrieving data

3. Data Layer (Database)

- SQLite database stores user details and prediction history
- Ensures structured and reliable data storage
- Maintains user-specific records for data isolation

4. Machine Learning Layer

- Random Forest model used for electricity bill prediction
- Processes appliance usage data (hours and counts)
- Generates predicted units and bill values
- Model stored as `.pkl` file and loaded during runtime

3.3 System Design Diagrams

3.3.1 Data Flow Diagram (DFD)

The Data Flow Diagram (DFD) illustrates the flow of data within the PowerPredict system. The user interacts with multiple core processes including User Management, Input Validation & Processing, ML Prediction Engine, and Tariff Bill Calculation.

This design ensures a clear and efficient flow of data across system components, integrating user interaction, machine learning prediction, and billing logic into a unified pipeline.

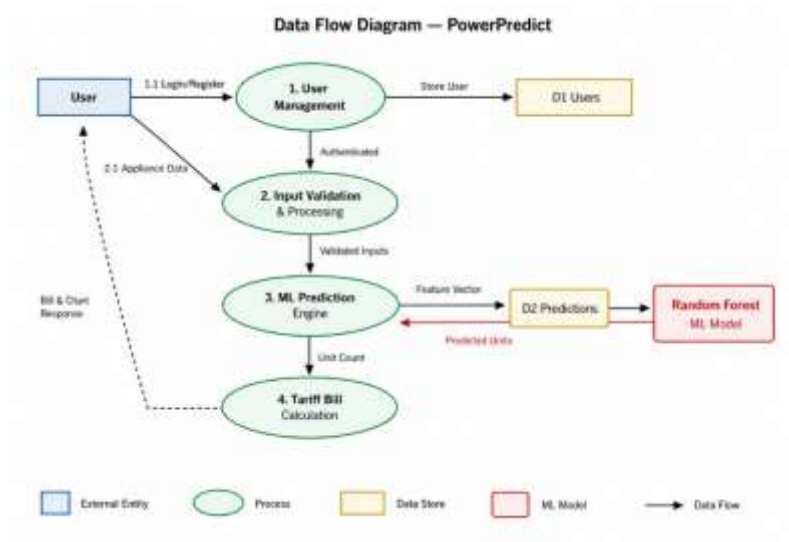


Fig 3.1 – Data Flow Diagram

3.3.2 Activity Diagram

Standard User Workflow:

The Activity Diagram for the Standard User Workflow begins with the user launching the application and proceeding to login. Upon successful authentication, the user accesses the dashboard and enters appliance usage details such as operating hours and appliance counts. The user then clicks the **Predict** button to initiate the prediction process. The system validates the inputs, processes the data, and sends it to the machine learning model for prediction. The system then stores the prediction in the database and displays the predicted bill and consumption results to the user.

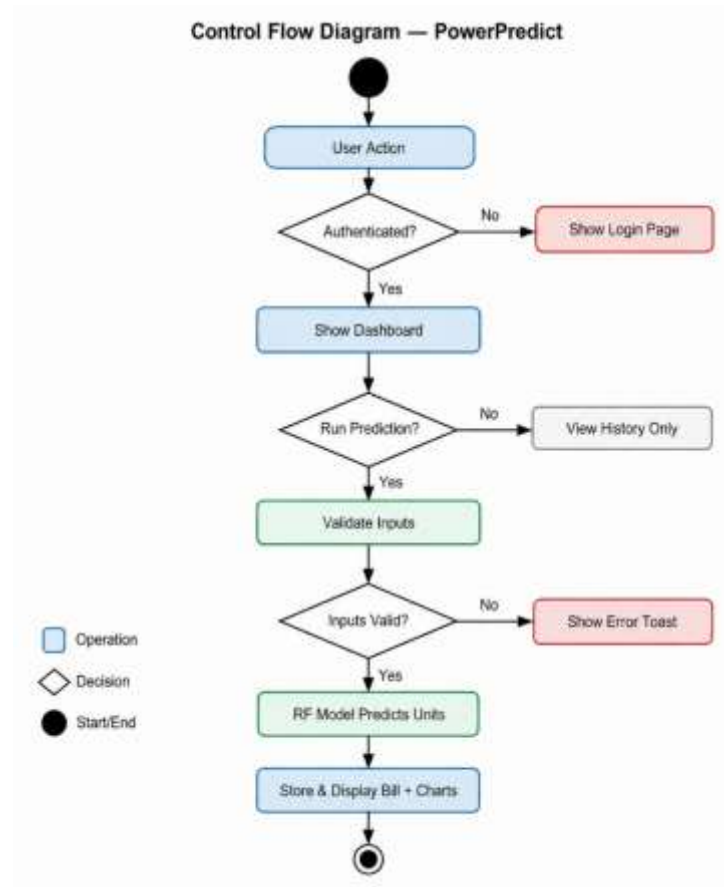


Fig 3.2 – Activity Diagram: Standard User Workflow

3.3.3 Use Case Diagram

The Use Case Diagram illustrates the interactions between different actors and the PowerPredict system. The primary actors are the User and the System Administrator.

This diagram highlights how users interact with the system to obtain predictions, while the administrator ensures smooth functioning and system reliability.

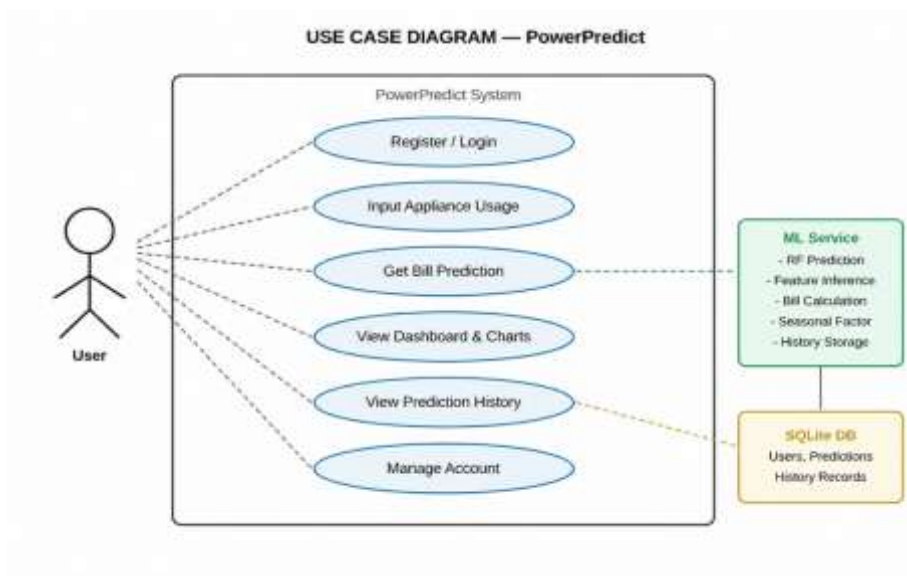


Fig 3.3 Use Case Diagram

3.3.3 ER Diagram (Entity Relationship Diagram)

The ER Diagram represents the database structure of the PowerPredict system. The primary entities are Users and Predictions.

The key relationship is: Users (1) to Predictions (many), where each user can generate and store multiple electricity bill predictions. Each prediction record includes appliance usage details, predicted units, predicted bill, and timestamp.

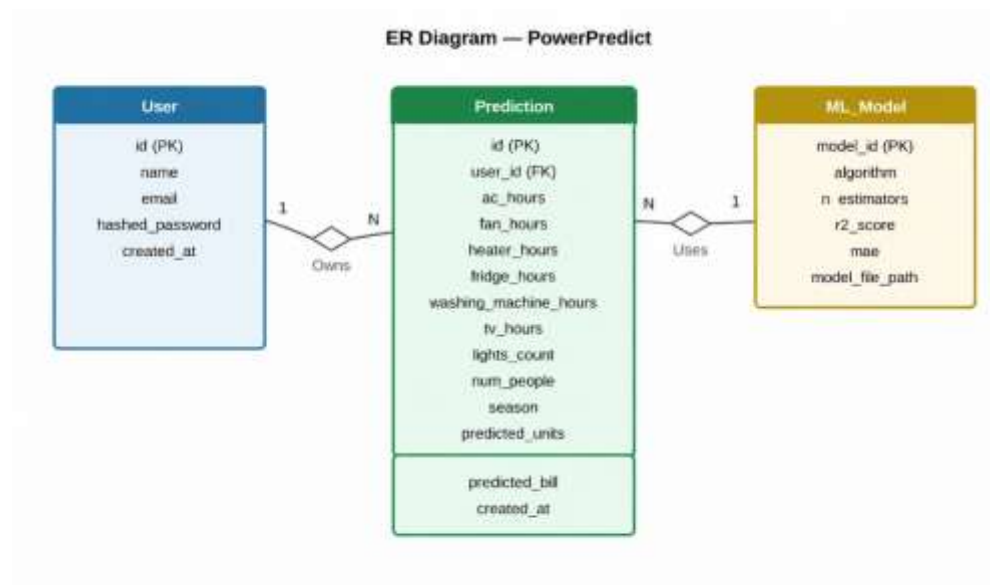


Fig 3.9 – Entity Relationship (ER) Diagram

3.4 Database Design

3.4.1 Logical Database Design

The logical design defines the structure of data using relational modeling principles. Key tables are:

- **Users table**
- **Predictions table**
- **Authentication table (JWT/session data)**

The **Users table** stores user credentials and profile details.

The **Predictions table** stores appliance usage inputs, predicted units, predicted bill, and timestamps, linked to users through a foreign key.

The authentication mechanism manages user sessions securely using token-based validation.

Chapter 4

Implementation, Testing and Maintenance

1.1 Introduction to Languages, IDEs, Tools and Technologies

The implementation phase of "**PowerPredict – AI-Based Electricity Bill Prediction System**" focuses on developing a fully functional application using modern web technologies and machine learning.

The backend is developed using **FastAPI**, a high-performance Python framework known for its speed and efficient API handling. FastAPI manages business logic including user authentication, input validation, data processing, and integration with the machine learning model. **SQLAlchemy** is used for database interaction, enabling smooth communication with the SQLite database.

For authentication and security, the system implements **JSON Web Token (JWT)** along with password hashing techniques. JWT ensures secure access to protected endpoints, while hashing protects user credentials.

The frontend is developed using **React.js**, utilizing functional components and hooks for state management. **Axios** is used for API communication between frontend and backend.

1.2 Testing Techniques and Test Plans

Testing ensures that the system functions correctly and meets all specified requirements.

Unit testing is performed on backend functions and machine learning components to verify correctness of individual modules such as input validation, prediction logic, and authentication handling.

API testing is conducted using Swagger UI and Postman to validate all endpoints. Each API is tested for correct request handling, proper responses, authentication enforcement, and error handling.

Integration testing verifies the interaction between the React frontend, FastAPI backend, SQLite database, and the machine learning model. The complete workflow—from user input to prediction generation and database storage—is tested.

1.3 Installation Instructions

The installation process involves setting up the FastAPI backend, SQLite database, machine learning model, and React.js frontend.

To run the backend, ensure Python 3.x is installed. Navigate to the backend folder and install dependencies using:

```
pip install -r requirements.txt
```

Start the server using:

```
uvicorn main:app --reload
```

The API will be available at: <http://127.0.0.1:8000/docs>

The SQLite database is automatically created when the backend runs, so no manual setup is required.

The machine learning model (`model.pkl`) should be placed in the appropriate folder and is loaded automatically by the backend during startup.

To run the frontend, ensure Node.js is installed. Navigate to the frontend folder and run:

```
npm install  
npm run dev
```

1.4 End User Instructions

The system is designed to provide a simple and user-friendly experience without requiring technical knowledge.

Upon launching the application, users can register or log in using their credentials. After successful login, the user is directed to the main dashboard.

Users can enter appliance usage details such as operating hours and appliance counts for devices like AC, fans, heater, refrigerator, washing machine, and television. After providing inputs, the user clicks the **Predict** button to generate the estimated electricity bill.

The system processes the input, displays predicted electricity units and bill amount, and stores the result in the database. Users can also view their previous predictions in the history section.

The application provides clear feedback for all actions and ensures smooth interaction throughout the process.

Chapter 5

Results and Discussion

5.1 User Interface Representation

The user interface of **PowerPredict** is developed using React.js, providing a modern, responsive, and user-friendly design. The interface is structured to allow easy input of appliance data and clear visualization of prediction results.

The Login and Registration screen provides a simple layout with input fields for name, email, and password. Upon successful authentication, users are redirected to the main dashboard.

The Dashboard serves as the primary interface where users can enter appliance usage details such as hours and appliance counts for devices like AC, fans, heater, refrigerator, washing machine, and television. A **Predict** button triggers the calculation process.

After submission, the system displays the predicted electricity units and estimated bill clearly. The interface also includes a section to view previous predictions stored in the database.

The application is designed to provide clear feedback messages and smooth navigation, ensuring a simple and effective user experience.

5.2 Description of System Modules

The system is divided into multiple modules, each handling a specific functionality, ensuring modularity and scalability.

The **Authentication Module** manages user registration, login, and secure session handling using JWT-based authentication and password hashing.

The **Input Processing Module** handles user inputs, validates appliance usage data, and prepares it for prediction.

The **Prediction Module** integrates the Random Forest machine learning model to estimate electricity consumption and billing based on input data.

The **Database Module** stores user information and prediction history using SQLite, ensuring data consistency and user-specific data access.

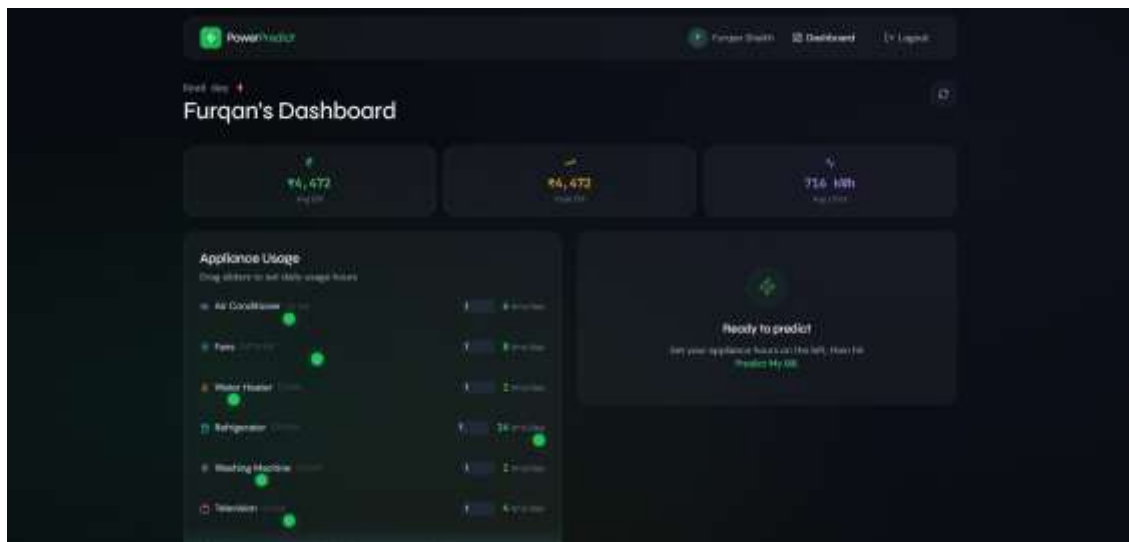
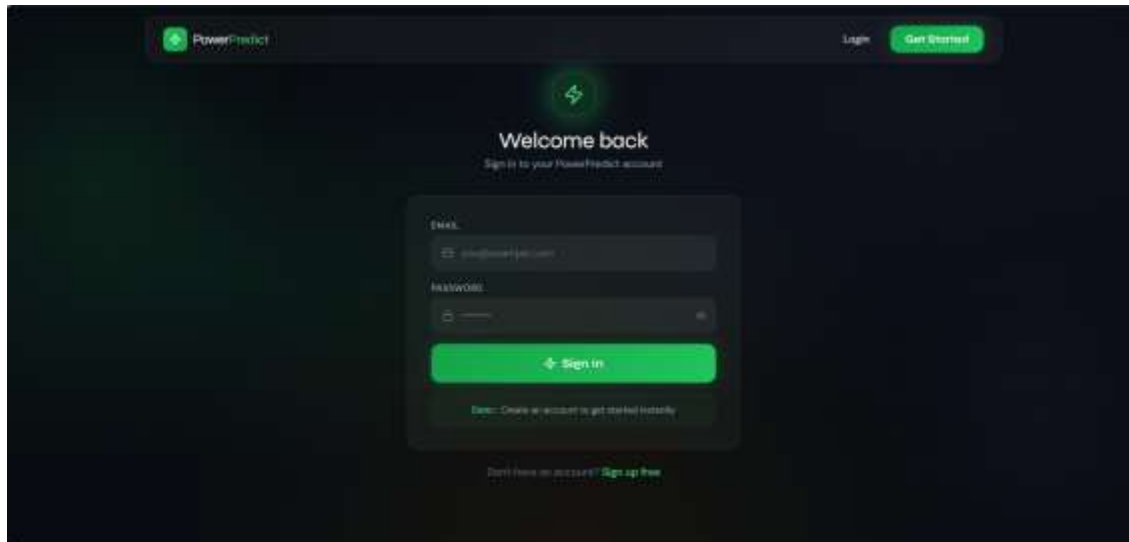
The **User Interface Module** provides an interactive frontend for input, result display, and navigation.

Each module communicates with the backend through RESTful APIs, ensuring efficient data flow and reliable system performance.

5.3 Snapshots of System

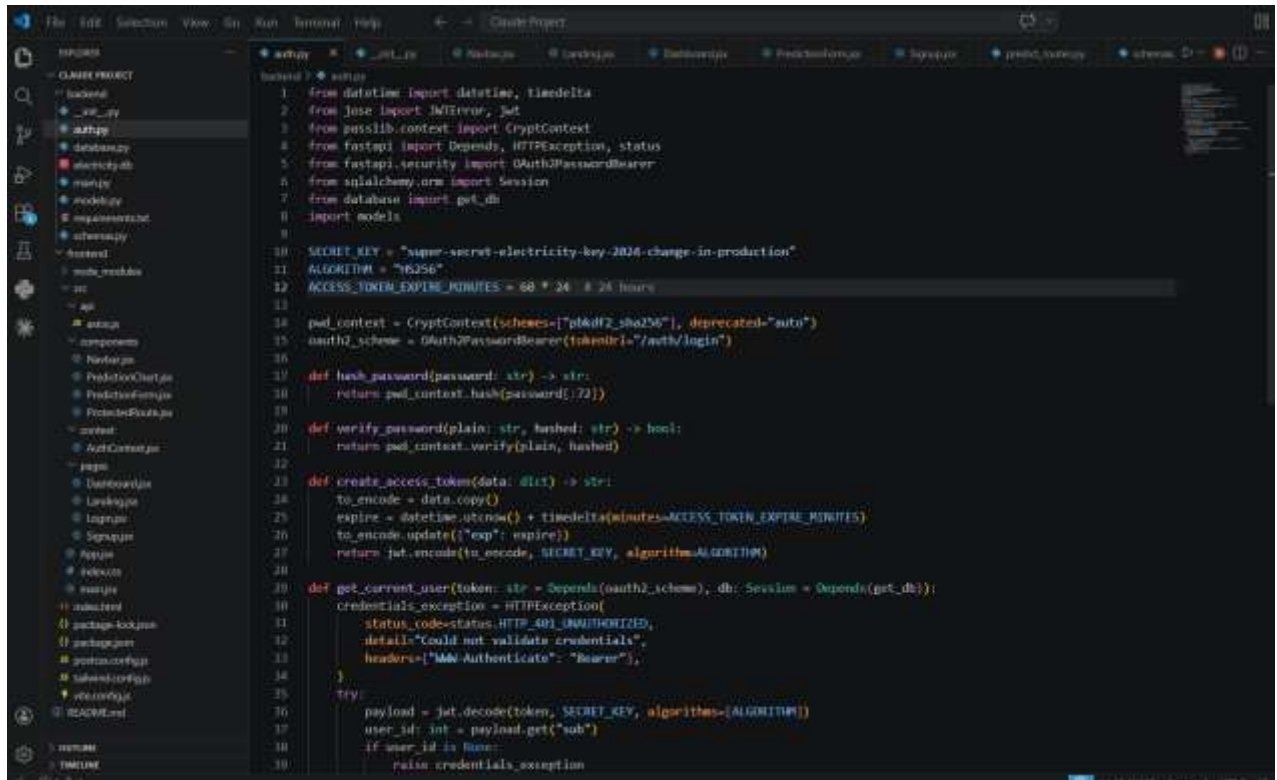
The system has been tested under various scenarios, and snapshots of application screens demonstrate its functionality. The login screen confirms successful authentication through secure user access. The dashboard allows users to input appliance usage details and initiate prediction.

The prediction result screen displays the estimated electricity units and bill clearly. The history section confirms successful storage of prediction data in the database. The interface provides real-time feedback and smooth navigation, ensuring effective interaction with the system.



5.4 Backend Representation

The backend is implemented using **FastAPI** and exposes a set of RESTful APIs. All APIs are documented and accessible through the built-in Swagger UI at /docs. The APIs handle operations related to user authentication, input processing, electricity bill prediction, and database storage of user data and prediction history.



```
1: from datetime import datetime, timedelta
2: from jose import JWTError, jwt
3: from passlib.context import CryptContext
4: from fastapi import Depends, HTTPException, status
5: from fastapi.security import OAuth2PasswordBearer
6: from sqlalchemy.orm import Session
7: from database import get_db
8: import models
9:
10: SECRET_KEY = "super-secret-electricity-key-2024-change-in-production"
11: ALGORITHM = "HS256"
12: ACCESS_TOKEN_EXPIRE_MINUTES = 60 * 24 * 7  # 7 days
13:
14: pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
15: oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/auth/login")
16:
17: def hash_password(password: str) -> str:
18:     return pwd_context.hash(password)
19:
20: def verify_password(plain: str, hashed: str) -> bool:
21:     return pwd_context.verify(plain, hashed)
22:
23: def create_access_token(data: dict) -> str:
24:     to_encode = data.copy()
25:     expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
26:     to_encode.update({"exp": expire})
27:     return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
28:
29: def get_current_user(token: str = Depends(oauth2_scheme), db: Session = Depends(get_db)):
30:     credentials_exception = HTTPException(
31:         status_code=status.HTTP_401_UNAUTHORIZED,
32:         detail="Could not validate credentials",
33:         headers={"WWW-Authenticate": "Bearer"},
34:     )
35:     try:
36:         payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
37:         user_id: int = payload.get("sub")
38:         if user_id is None:
39:             raise credentials_exception
```

The Python Intelligence Engine is invoked by the Spring Boot backend when insights, predictions, or behavioral analysis are requested. The engine processes transaction data and returns structured JSON results consumed by the backend service layer.

5.5 Database Representation

The database used in this system is **SQLite**, which stores all relevant data in structured tables. The main tables include **Users** and **Predictions**.

The **Users table** stores registered user credentials with securely hashed passwords. The **Predictions table** stores appliance usage inputs, appliance counts, predicted electricity units, predicted bill, and timestamps for each prediction.

Referential integrity is maintained through foreign key relationships between users and their predictions. The database ensures consistent data storage and retrieval, allowing users to access their prediction history efficiently.

Chapter 6

Summary and Conclusions

6.1 Summary

PowerPredict – AI-Based Electricity Bill Prediction System is a full-stack web application developed to provide intelligent and proactive electricity cost estimation. The system integrates a React.js frontend, FastAPI backend, SQLite database, and a Random Forest machine learning model.

The project involved requirement analysis, system design, backend development for API handling and authentication, and frontend development for user interaction. The system allows users to input appliance usage data, generates real-time electricity bill predictions, and stores results for future reference.

Various testing techniques such as unit testing, API testing, and system testing were performed to ensure accuracy and reliability. The application was validated across different scenarios including authentication, prediction processing, and data storage.

Overall, the project demonstrates the effective integration of full-stack web development with machine learning to solve a real-world problem.

6.2 Conclusions

The project successfully achieves its objective of providing an intelligent platform for electricity bill prediction. It enables users to estimate their electricity costs in advance and understand the impact of appliance usage.

The use of FastAPI ensures efficient backend performance, while SQLite provides reliable data storage. The Random Forest model enables accurate prediction based on user inputs. Secure authentication ensures safe access to the system.

The React.js frontend provides a simple and interactive interface, making the system easy to use for non-technical users.

In conclusion, PowerPredict serves as a practical and scalable solution that combines web development and machine learning, with strong potential for future enhancements such as real-time data integration and advanced analytics.

Chapter 7

Future Scope

7.1 Model Improvement

The current system uses a Random Forest model for prediction. Future work can include advanced machine learning or deep learning models to improve accuracy and adapt to real-world usage patterns.

7.2 Cloud Deployment and Scalability

The system can be deployed on cloud platforms such as AWS or Azure for remote access, better scalability, and improved performance.

7.3 Real-Time Data Integration

Integration with smart meters or IoT devices can enable automatic data collection, eliminating manual input and improving accuracy.

7.4 Mobile Application Development

A mobile application for Android and iOS can provide easy access, real-time predictions, and notifications.

7.5 Security Enhancements

Future versions can include advanced security features such as multi-factor authentication and enhanced data protection mechanisms.

7.6 Advanced Analytics

The system can be extended to provide detailed consumption analytics, appliance-level insights, and personalized energy-saving recommendations.

7.7 Conclusion of Future Scope

PowerPredict provides a strong foundation for intelligent electricity management. With enhancements like advanced ML models, cloud deployment, and real-time data integration, it can evolve into a scalable and practical solution for real-world energy optimization.

Appendix A — API Endpoints

- `POST /auth/login`
- `POST /predict`
- `GET /history`
- `GET /users/me`

Bibliography

1. [1] Sommerville, I., *Software Engineering*, Pearson Education, 10th Edition, 2015.
2. [2] FastAPI Documentation, *FastAPI Framework*, Available: <https://fastapi.tiangolo.com/>, Accessed: 2026.
3. [3] React.js Documentation, *React – A JavaScript Library for Building User Interfaces*, Available: <https://react.dev>, Accessed: 2026.
4. [4] SQLite Documentation, *SQLite Database Engine*, Available: <https://www.sqlite.org/docs.html>, Accessed: 2026.
5. [5] McKinney, W., *Python for Data Analysis*, O'Reilly Media, 2nd Edition, 2018.
6. [6] JWT.io, *JSON Web Tokens – Introduction*, Available: <https://jwt.io/>, Accessed: 2026.
7. [7] Pressman, R. S., *Software Engineering: A Practitioner's Approach*, McGraw-Hill, 7th Edition.
8. [8] Axios Documentation, *Promise Based HTTP Client for Node.js*, Available: <https://axios-http.com/docs/intro>, Accessed: 2026.
9. [9] Scikit-learn Documentation, *Machine Learning in Python*, Available: <https://scikit-learn.org/stable/>, Accessed: 2026.
10. [10] Pandas Documentation, *pandas – Python Data Analysis Library*, Available: <https://pandas.pydata.org/docs/>, Accessed: 2026.

PowerPredict: ML-Based Electricity Bill Prediction System Using Machine Learning

A Synopsis

for

Minor Project

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

Mohammed Furqan Shaikh [EN23CS301626]

Minhaj Ahmed Sheikh [EN23CS301616]

Mo. Fazil Ahmed Ghori [EN23CS301618]

Under the Guidance of

**Prof. Sheetal Bawane &
Prof. Garima Silakari Tukra**



MEDICAPS
UNIVERSITY

**Department of Computer Science & Engineering Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE - 453331**

Jan - June 2026

1. Introduction

The increasing cost of electricity has become a significant concern for households, students, and small businesses. With the growing use of electrical appliances and modern technologies, managing electricity consumption efficiently has become more challenging. Despite this, most users only become aware of their electricity expenses after receiving the monthly bill, leaving them with limited opportunity to control excessive energy usage. This lack of real-time insight creates a gap between consumption and awareness, resulting in inefficient energy management.

Traditional methods of estimating electricity bills rely on manual calculations or rough assumptions based on previous usage patterns. These approaches are often inaccurate and fail to consider variations in usage behavior, seasonal changes, and appliance consumption. As a result, users are unable to make informed decisions about their energy usage. There is a clear need for a system that can provide accurate and timely predictions of electricity bills based on user-specific consumption patterns.

Machine learning offers a powerful solution to this problem by enabling systems to learn patterns from historical data and make accurate predictions. By analyzing relationships between input features such as appliance usage and electricity consumption, machine learning models can estimate future electricity bills with high accuracy. The proposed project, PowerPredict, builds on this approach to develop an intelligent electricity bill prediction system that helps users understand and manage their energy consumption effectively.

The system is developed using a full-stack architecture. A FastAPI-based backend handles data processing and communication with the machine learning model, while a React-based frontend provides a modern and user-friendly interface for input and result visualization. The machine learning component is implemented using a Random Forest regression model trained on structured electricity consumption data. This architecture ensures efficient interaction between components while maintaining scalability and performance.

Usability and performance are key aspects of the system design. The application is designed to be accessible to non-technical users, allowing them to easily input usage data and obtain predictions instantly. The backend ensures fast response times, while the frontend provides a clean and intuitive interface. The system also incorporates validation mechanisms to ensure reliable input handling and accurate predictions.

The expected outcome is a fully functional electricity bill prediction system that demonstrates the practical application of machine learning in solving real-world problems. The project highlights the integration of artificial intelligence with full-stack web development to create a

scalable, efficient, and user-friendly solution for electricity cost forecasting and energy management.

2. Literature Review

Electricity consumption prediction and energy management systems have been widely studied in both academic research and industry applications. Early work in this domain focused on statistical and regression-based techniques for estimating energy usage. Qureshi et al. [1] demonstrated the effectiveness of machine learning models in predicting residential electricity consumption, highlighting the importance of data-driven approaches over traditional estimation methods.

Pedregosa et al. [2] introduced the scikit-learn library, which has become a widely used framework for implementing machine learning algorithms in Python. It provides efficient tools for regression, classification, and model evaluation, enabling the development of predictive systems such as electricity bill estimation. Géron [3] further explored practical applications of machine learning techniques, including ensemble methods like Random Forest, which are known for their robustness and high prediction accuracy.

Recent studies have explored various machine learning models for energy prediction. Linear regression models provide a simple baseline but often fail to capture non-linear relationships in consumption data. More advanced models such as Random Forest and Gradient Boosting have shown improved performance due to their ability to model complex interactions between input features. Random Forest, in particular, has been widely adopted due to its balance between accuracy, interpretability, and resistance to overfitting.

From an application perspective, existing electricity prediction systems often suffer from several limitations. Many systems rely on static estimation methods and do not adapt to user-specific consumption patterns. Additionally, some solutions lack user-friendly interfaces, making them inaccessible to non-technical users. Others fail to provide real-time predictions, limiting their usefulness for proactive energy management.

The use of modern web technologies has enabled the development of interactive and scalable prediction systems. Frameworks such as FastAPI provide high-performance backend support for real-time API communication, while React enables the creation of responsive and user-friendly interfaces. These technologies, when combined with machine learning models, allow for seamless integration of prediction systems into practical applications.

Existing systems exhibit several limitations when applied to real-world electricity bill prediction:

- Limited accuracy due to reliance on simple statistical models

- Lack of personalization based on individual usage patterns
- Absence of real-time prediction capabilities
- Poor user interface design reducing accessibility
- Limited scalability and integration with modern web technologies

Compared to existing solutions, the proposed PowerPredict system offers:

- Accurate bill prediction using Random Forest regression
- Real-time prediction through FastAPI-based backend services
- User-friendly interface developed using React
- Scalable architecture supporting future enhancements
- Practical applicability for household electricity consumption management

Thus, the proposed system addresses a significant gap by providing a complete, integrated solution that combines machine learning with modern web technologies to deliver accurate, real-time, and user-friendly electricity bill prediction for everyday users.

3. Problem Definition

Estimating electricity bills accurately is a challenge faced by households, students, and small businesses due to the lack of accessible and reliable prediction tools. Most users rely on past bills or rough estimates, which do not account for variations in appliance usage, seasonal changes, or consumption behavior. As a result, users often receive unexpectedly high electricity bills without having the opportunity to control their energy usage in advance.

Existing methods attempt to address this problem through manual calculations or basic estimation tools, but these approaches introduce several limitations such as inaccuracy, lack of personalization, and limited usability. Many systems do not provide real-time insights, making them ineffective for proactive energy management.

The major problems identified are:

- Lack of accurate prediction — users cannot reliably estimate their monthly electricity bills
- Absence of personalization — existing methods do not adapt to individual usage patterns
- No real-time estimation — predictions are not available before the billing cycle ends
- Limited user awareness — users cannot identify how their usage affects electricity costs
- Poor accessibility — many tools are not user-friendly for non-technical users
- Inconsistent estimation methods — results vary due to lack of data-driven approaches
- Lack of scalable solutions — existing systems do not integrate modern web and AI technologies

These issues result in inefficient energy usage and financial uncertainty for users, particularly for those with limited awareness of electricity consumption patterns. Unexpected high bills can lead to difficulty in budgeting and managing expenses.

Therefore, there is a need for a system that can accurately predict electricity bills based on user-specific inputs, provide real-time insights, and enable users to make informed decisions about their energy consumption using data-driven approaches.

4. Objectives

The primary objective of this project is to develop a functional electricity bill prediction system that enables users to estimate their monthly electricity expenses accurately using machine learning and modern web technologies.

Specific Objectives:

- To design and implement a machine learning model using Random Forest regression for accurate electricity bill prediction
- To preprocess and structure input data such as electricity usage and consumption patterns for effective model training
- To train and evaluate the model using appropriate regression metrics such as MAE, RMSE, and R^2 score
- To develop a FastAPI-based backend that handles API requests, input validation, and real-time prediction processing
- To build a responsive React-based frontend providing an intuitive user interface for entering usage data and viewing predictions
- To integrate frontend and backend components using RESTful APIs for seamless data communication
- To implement a user authentication system for secure login and personalized access to the application
- To ensure fast response time for prediction requests to support real-time user interaction
- To design the system architecture to support scalability and future enhancements such as additional features and improved models
- To document all system design decisions, implementation details, and evaluation results to academic standard

5. Methodology

The proposed system, PowerPredict, follows a structured and systematic Software Development Life Cycle (SDLC) approach to ensure organised planning, efficient development, continuous

validation, and reliable deployment. The adopted methodology emphasises modular design, iterative development, and data-driven implementation to achieve an accurate, scalable, and maintainable system.

The development process is divided into the following phases:

Phase 1: Requirement Analysis

The requirement analysis phase focuses on identifying, documenting, and validating all system requirements before development begins.

Activities Performed:

I. Identification of Functional Requirements:

- User input interface for electricity consumption details
- Machine learning-based electricity bill prediction
- Real-time prediction through API requests
- User authentication for secure login and access
- Display of predicted electricity bill on dashboard
- Input validation to ensure correct and reliable data
- Integration between frontend, backend, and ML model

II. Identification of Non-Functional Requirements:

- Performance: API response time under 500 milliseconds
- Accuracy: High prediction accuracy using Random Forest regression
- Usability: Simple and intuitive interface for non-technical users
- Scalability: Ability to handle multiple users and future feature expansion
- Reliability: Stable model inference and consistent prediction output

III. Stakeholder Identification:

- Users: Individuals or households predicting electricity bills
 - Developers: Responsible for system design and implementation
 - Administrators: Managing system deployment and maintenance
-

Phase 2: System Design

In this phase, conceptual requirements are transformed into a structured technical design defining system implementation.

I. Architectural Design

The system follows a three-layer architecture:

- Frontend Layer: React-based user interface for data input and result display
- Backend Layer: FastAPI server handling API requests and model interaction
- Machine Learning Layer: Random Forest model for prediction

Communication between frontend and backend is handled through RESTful APIs, ensuring smooth and efficient data exchange.

II. Machine Learning Model Design

The system uses a Random Forest regression model trained on structured electricity consumption data. The dataset is split into training and testing sets, and the model is evaluated using regression metrics such as MAE, RMSE, and R^2 score.

III. Data Handling and Storage

The trained model is stored as a serialized `.pkl` file and loaded during runtime. Input data is processed dynamically and not permanently stored, ensuring lightweight system operation.

IV. API Design

The backend exposes REST API endpoints for prediction:

- POST `/predict` — receives user input and returns predicted electricity bill

The API ensures proper validation, structured responses, and efficient processing.

V. User Interface Design

The user interface is designed for simplicity and accessibility. Key screens include:

- Landing Page — introduction and navigation
 - Authentication Pages — login and signup
 - Dashboard — input fields and prediction results
-

Phase 3: Machine Learning Development

Key Activities:

- Dataset creation and preprocessing
 - Feature selection based on electricity consumption patterns
 - Training Random Forest regression model using scikit-learn
 - Model evaluation using MAE, RMSE, and R^2 metrics
 - Saving trained model as `.pkl` file for deployment
-

Phase 4: Backend Development

Key Activities:

- Setting up FastAPI backend framework
 - Implementing `/predict` API endpoint
 - Integrating machine learning model with backend
 - Handling JSON request and response processing
 - Implementing input validation and error handling
 - Ensuring fast and efficient prediction response
-

Phase 5: Frontend Development

Key Activities:

- Developing React-based user interface
 - Creating input forms for electricity usage
 - Implementing API integration with backend
 - Displaying prediction results dynamically
 - Handling user authentication and navigation
 - Ensuring responsive and user-friendly design
-

Phase 6: Integration and Testing

Testing Activities:

i. Unit Testing:

- Testing individual components such as model prediction and API functionality

ii. API Testing:

- Validating API endpoints using tools like Postman
- Ensuring correct request-response handling

iii. Integration Testing:

- Verifying communication between frontend, backend, and ML model
- Testing complete prediction workflow

iv. System Testing:

- Testing system with different input scenarios
- Ensuring accurate and consistent predictions

v. Performance Testing:

- Measuring API response time
- Ensuring smooth frontend interaction

6. References

- [1] T. A. Qureshi, A. Nair, and M. Faruque, “Prediction of Energy Consumption in Residential Buildings Using Machine Learning,” IEEE Access, 2020.
- [2] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, 2011.
- [3] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed., O’Reilly Media, 2022.
- [4] L. Breiman, “Random Forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.
- [5] Ministry of Power, Government of India, “Annual Report on Power Sector,” 2023.
- [6] FastAPI Documentation. Available: <https://fastapi.tiangolo.com/>
- [7] React Documentation. Available: <https://react.dev/>
- [8] Python Software Foundation, “Python Documentation.” Available: <https://docs.python.org/>
- [9] S. Raschka and V. Mirjalili, *Python Machine Learning*, 3rd ed., Packt Publishing, 2019.
- [10] J. Brownlee, *Machine Learning Mastery with Python*, Machine Learning Mastery, 2020.

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements for the PowerPredict Electricity Bill Prediction System (PEBPS), Version 1.0. The document covers the complete system including the machine learning prediction module developed using Python and scikit-learn, the FastAPI-based backend, and the React-based web frontend. This SRS serves as the contractual baseline between the development team and the faculty evaluators, defining what the system shall do and the constraints under which it shall operate.

This document covers the full scope of the PEBPS as a browser-based application for electricity bill prediction. It does not include integration with real smart meter infrastructure or external utility billing systems, which are considered potential future enhancements.

1.2 Document Conventions

Requirements in this document are written using the keyword SHALL for mandatory requirements, SHOULD for recommended requirements, and MAY for optional requirements. Requirement identifiers follow the format FR-X.Y for functional requirements and NFR-X for non-functional requirements, where X represents the section number and Y represents the sequence number within that section. Priority levels are defined as High (H), Medium (M), and Low (L). All performance values are expressed as maximum limits under worst-case conditions unless otherwise specified. Technical terminology used in this document is defined in Appendix A.

1.3 Intended Audience and Reading Suggestions

This document is intended for multiple categories of readers. Faculty evaluators and project supervisors should review Sections 1 through 4 to understand the system scope, architecture, and functional behavior, while Section 5 defines the quality and performance expectations. Developers implementing the system should read all sections in sequence, with particular emphasis on Section 4 for functional requirements and system interactions. Testers should focus on Section 4 for deriving test cases and Section 5 for validation criteria. Documentation writers may use Sections 2 and 3 as a reference for preparing user guides and system documentation.

1.4 Product Scope

The PowerPredict Electricity Bill Prediction System is a web-based machine learning application designed to help users estimate their monthly electricity bill before the end of the billing cycle. By accepting user input related to electricity consumption and usage patterns, and applying a trained Random Forest regression model, the system provides an accurate and real-time bill prediction.

The product focuses on delivering a simple and user-friendly interface for entering input data and viewing prediction results. It is intended for use by households, students managing shared living expenses, and small-scale users who wish to monitor and control electricity costs. The system is designed to be scalable and extensible, allowing future integration of advanced features such as real-time data sources, enhanced prediction models, and mobile-based access.

2. Overall Description

2.1 Product Perspective

The PowerPredict Electricity Bill Prediction System is a new, self-contained web application with no predecessor product. It operates as a standalone platform that accepts electricity usage inputs through a browser interface, processes them through a FastAPI-based backend, and returns a bill prediction using a pre-trained Random Forest regression model. The product has no dependency on external payment systems, blockchain infrastructure, or utility provider APIs.

The system consists of three major subsystems: the machine learning module (a trained Random Forest model stored as a `.pkl` file) providing bill predictions; the FastAPI backend providing REST API services, input validation, and model inference; and the React-based frontend providing the user interface with real-time result display. From the user's perspective, the application operates as a single-page web interface accessible through any modern browser without installation.

2.2 Product Functions

At a high level, the PowerPredict system performs the following major functions: collection of electricity usage inputs through a user-friendly web interface; validation and processing of input data; prediction of monthly electricity bill using a trained Random Forest regression model; real-time communication between frontend and backend through API requests; and display of predicted results in a clear and understandable format for users.

2.3 User Classes and Characteristics

Household Users (Primary User Class): Individuals or families who want to estimate their monthly electricity bill before it arrives. These users are familiar with basic web applications but have no technical knowledge of machine learning. They focus on ease of use and prediction accuracy. This group represents approximately 70% of expected users.

Students and Shared Accommodation Residents (Secondary User Class): Students or individuals sharing living spaces who need to estimate and manage electricity expenses. These users are digitally literate and interested in understanding how their usage affects the total bill. This group represents approximately 20% of expected users.

Small Business Owners (Secondary User Class): Small-scale users such as shop owners or office managers who require approximate electricity cost estimation for budgeting purposes. These users prioritize quick results and simplicity. This group represents approximately 10% of expected users.

2.4 Operating Environment

The system shall operate on any modern web browser (Chrome 90+, Firefox 88+, Edge 90+, Safari 14+) without requiring additional plugins or extensions. The backend server shall run on Python 3.10+ using FastAPI and can be deployed on Linux, Windows, or macOS environments. The machine learning model shall be loaded from a `.pkl` file during server startup using joblib, and no specialized hardware such as GPUs is required. The system can run locally or be deployed on cloud platforms such as Render or Railway for remote access.

2.5 Design and Implementation Constraints

The system shall be implemented using Python for backend and machine learning components, with FastAPI as the web framework and scikit-learn for model training and inference. The frontend shall be developed using React for building a responsive and interactive user interface. The trained model shall be serialized using joblib and loaded at backend startup. The system shall not rely on external APIs or real-time data sources and shall function independently based on user-provided inputs. User authentication may be implemented for controlled access, but persistent storage of user data is limited to essential functionality.

2.6 User Documentation

The system shall provide basic user documentation including input field descriptions and usage instructions within the application interface. A brief guide explaining how to use the system (enter data, submit input, view prediction) shall be included on the main interface. Additionally, a developer README file shall document project setup, required dependencies, and steps to run the

backend and frontend. Troubleshooting guidance shall be provided for common issues such as missing model files, server startup errors, and API connection problems.

2.7 Assumptions and Dependencies

The system assumes that the machine learning model has been trained and saved before the application is executed; absence of the model file will result in backend failure. The system depends on compatible Python libraries as specified in the project requirements; mismatched versions may lead to runtime errors. The dataset used for training is assumed to represent typical electricity consumption patterns, and prediction accuracy may vary depending on real-world usage differences. The system also assumes stable interaction between frontend and backend services for correct functionality.

3. External Interface Requirements

3.1 User Interfaces

The application shall present a responsive single-page web interface accessible on desktop and tablet devices. The interface consists of the following primary sections:

Input Form: A structured layout allowing users to enter electricity usage details such as consumption units and usage-related parameters. All input fields include descriptive labels, default values representing typical usage, and valid range constraints to ensure correct data entry.

Predict Button: A primary action button that submits form data to the FastAPI backend and triggers the prediction process. The button shall be disabled during API execution to prevent duplicate requests.

Results Section: Displayed below the input form after prediction completion. It shows the predicted monthly electricity bill in rupees as a highlighted output card. The results are presented in a clear and user-friendly format for easy interpretation.

3.2 Hardware Interfaces

The system does not directly interact with specialized hardware components. It operates using standard computing devices such as desktops, laptops, tablets, and smartphones. User input is provided through keyboard, mouse, or touch interfaces, and output is displayed using the browser's standard rendering system. The system does not interface with smart meters, IoT devices, or external hardware sensors. Communication between frontend and backend occurs through standard network protocols over HTTP or HTTPS.

3.3 Software Interfaces

FastAPI REST API (Backend Layer): The frontend communicates with the FastAPI backend through a POST `/predict` endpoint, sending user input data in JSON format. The backend processes the input and returns a JSON response containing the predicted electricity bill.

scikit-learn (Machine Learning Layer): The backend loads the trained Random Forest regression model (`model.pkl`) at application startup using `joblib`. During prediction, input data is formatted appropriately and passed to the model for inference.

Pandas and NumPy (Data Processing Layer): The backend uses Pandas for structuring input data and ensuring consistency in feature format. NumPy is used for numerical operations required during prediction processing.

3.4 Communications Interfaces

The frontend communicates with the FastAPI backend over HTTP during development (e.g., localhost) and HTTPS in production environments. All data exchange is performed using UTF-8 encoded JSON. The system operates in a stateless manner, where each prediction request is independent and does not rely on stored session data. No external analytics or third-party services are used for data transmission. Cross-Origin Resource Sharing (CORS) settings shall be configured on the backend to allow secure communication between frontend and backend applications.

4. System Features and Functional Requirements

4.1 FR1: Appliance Usage Input Collection

4.1.1 Description and Priority

The system shall provide a structured web form collecting all required appliance usage and household parameters necessary for bill prediction. Priority: High.

4.1.2 Functional Requirements

FR-1.1: The system **SHALL** accept the following appliance usage inputs as numeric values: AC usage hours per day (0–12), heater usage hours per day (0–8), TV usage hours per day (0–10), number of refrigerators (1 or 2), and washing machine presence (boolean toggle).

FR-1.2: The system **SHALL** accept the following household parameters: number of rooms (1–6), number of occupants (1–8), average monthly temperature in °C (10–50), current season (Summer, Winter, Monsoon, Spring), location type (Urban, Semi-Urban, Rural), and electricity tariff slab (Low, Medium, High).

FR-1.3: The system **SHALL** apply client-side validation to all numeric inputs, rejecting values outside the defined ranges with an inline error message before the form is submitted.

FR-1.4: The system **SHALL** pre-populate all input fields with representative default values reflecting average Indian household usage, so a user can click Predict immediately without entering any data.

FR-1.5: The system **SHALL** disable the Predict button during an active API request and re-enable it upon response receipt or request failure, preventing duplicate submissions.

4.2 FR2: Feature Engineering and Input Preprocessing

4.2.1 Description and Priority

The system shall apply the same feature engineering pipeline used during model training to transform raw user inputs into the 13-feature vector expected by the XGBoost model. Priority: High.

4.2.2 Functional Requirements

FR-2.1: The system **SHALL** encode the categorical inputs season, location, and tariff_slab to integer values using the persisted LabelEncoder instances loaded at application startup, using the same mapping applied during training.

FR-2.2: The system **SHALL** compute the derived feature appliance_load using the formula: $(ac_hours \times 1.5) + (heater_hours \times 2.0) + (tv_hours \times 0.1) + (fridge_count \times 40 / 30)$.

FR-2.3: The system **SHALL** compute the derived feature temp_occupant_index using the formula: $avg_monthly_temp \times num_occupants$.

FR-2.4: The system **SHALL** assemble the 13-feature input array in the exact column order stored in features.pkl, ensuring the model receives features in the order it was trained on.

FR-2.5: The system **SHALL** reject any prediction request where one or more required input fields are missing or contain non-numeric values, returning an HTTP 400 response with a descriptive error message.

4.3 FR3: Bill Prediction

4.3.1 Description and Priority

The system shall invoke the trained XGBoost model to generate a monthly electricity bill prediction from the preprocessed feature vector. Priority: High.

4.3.2 Functional Requirements

FR-3.1: The system **SHALL** load the trained XGBoost model from `best_model.pkl` at Flask application startup using `joblib`, and keep it resident in memory for the lifetime of the server process to avoid per-request loading overhead.

FR-3.2: The system **SHALL** invoke `model.predict()` on the assembled 13-feature Pandas DataFrame and return the predicted bill amount in Indian Rupees, rounded to the nearest integer.

FR-3.3: The system **SHALL** calculate the estimated monthly kWh consumption by dividing the predicted bill by the tariff rate corresponding to the user's selected tariff slab (Low: ₹4.5/kWh, Medium: ₹6.0/kWh, High: ₹8.5/kWh).

FR-3.4: The system **SHALL** return both the predicted bill in rupees and the estimated kWh consumption in the prediction response JSON object.

FR-3.5: The system **SHALL** return an HTTP 500 response with a descriptive error message if model inference fails for any reason, without exposing internal stack traces to the frontend.

4.4 FR4: Slab-Based Tariff Application

4.4.1 Description and Priority

The system shall apply Madhya Pradesh Electricity Board's slab-based tariff structure to align predicted bill amounts with actual utility billing practice. Priority: High.

4.4.2 Functional Requirements

FR-4.1: The system **SHALL** maintain a configurable tariff rate table mapping the three tariff slab categories (Low, Medium, High) to their respective per-unit rates in rupees per kWh.

FR-4.2: The system **SHALL** apply the tariff rate corresponding to the user's selected slab when computing the kWh estimate from the model's rupee output, ensuring the kWh figure is consistent with the selected tariff tier.

FR-4.3: The system **SHALL** display the applied tariff slab and the corresponding rate per unit alongside the prediction result, so the user understands the basis of the calculation.

FR-4.4: The system **SHALL** allow the tariff rate table to be updated by modifying a single configuration constant in the backend code, without requiring changes to any other module.

4.5 FR5: Per-Appliance Consumption Breakdown

4.5.1 Description and Priority

The system shall compute and display a per-appliance monthly kWh consumption breakdown, enabling users to identify which appliances are contributing most to their predicted bill. Priority: Medium.

4.5.2 Functional Requirements

FR-5.1: The system **SHALL** calculate the monthly kWh contribution of each appliance using its rated wattage and the user-entered daily usage hours, multiplied by 30 days.

FR-5.2: The system **SHALL** compute per-appliance kWh values for: AC (1500W), Heater (2000W), Refrigerator (40 kWh/month fixed), TV (100W), Washing Machine (30 kWh/month fixed), and an Other category comprising remaining base consumption.

FR-5.3: The system **SHALL** generate a pie chart using Matplotlib showing the proportional kWh contribution of each appliance category, encoded as a base64 PNG string and included in the prediction API response.

FR-5.4: The system **SHALL** display the pie chart in the results section of the frontend upon successful prediction, with appliance labels and percentage values visible on or adjacent to each pie segment.

4.6 FR6: Personalized Energy-Saving Tips

4.6.1 Description and Priority

The system shall generate personalized energy-saving recommendations based on the user's specific appliance usage profile, targeting the highest-consuming appliances. Priority: Medium.

4.6.2 Functional Requirements

FR-6.1: The system **SHALL** evaluate each appliance's computed kWh contribution against a defined threshold and generate a tip for any appliance whose usage exceeds that threshold.

FR-6.2: The system **SHALL** generate the following condition-based tips: AC tip if `ac_hours > 6` (reduce AC usage by 2 hours per day); heater tip if `heater_hours > 3` (use a thermostat timer); fridge tip if `fridge_count > 1` (consolidate to one energy-efficient unit); tariff tip if `tariff_slab` is High (shift usage to off-peak hours); and a general smart meter tip for all users.

FR-6.3: The system **SHALL** include at least one tip in every prediction response, defaulting to the general smart meter tip if no appliance-specific thresholds are exceeded.

FR-6.4: The system **SHALL** display the tips as a clearly formatted list in the results section, with each tip preceded by a relevant emoji icon for visual clarity.

4.7 FR7: Prediction Results Display

4.7.1 Description and Priority

The system shall display all prediction outputs in a clearly structured results section that renders without requiring a page reload.

4.7.2 Functional Requirements

FR-7.1: The system **SHALL** display the predicted monthly bill as a prominently styled metric card showing the rupee value in a large font size, labelled Predicted Monthly Bill.

FR-7.2: The system **SHALL** display the estimated monthly consumption as a second metric card showing the kWh value, labelled Estimated Consumption.

FR-7.3: The system **SHALL** render the results section dynamically upon receiving the prediction response without a full page reload, using JavaScript to update the DOM in place.

FR-7.4: The system **SHALL** display a loading indicator on the Predict button from the moment the request is sent until the response is fully received and rendered.

FR-7.5: The system **SHALL** display a user-friendly error message in the results section if the prediction API request fails, without showing raw error objects or HTTP status codes.

FR-7.6: The system **SHALL** allow the user to modify any input and submit a new prediction without reloading the page, with the results section updating to reflect the new prediction.

4.8 FR8: Model Training Pipeline

4.8.1 Description and Priority

The system shall provide a reproducible model training pipeline that generates all required .pkl artifacts from the dataset, producing a deployable model without manual intervention. Priority: High.

4.8.2 Functional Requirements

FR-8.1: The system **SHALL** train four regression models — Linear Regression, Ridge Regression, Random Forest Regressor, and XGBoost Regressor — on the same training split and evaluate all four on the same test split.

FR-8.2: The system **SHALL** select the best-performing model based on R^2 score on the test split and persist it to models/best_model.pkl using joblib.

FR-8.3: The system **SHALL** persist the three fitted LabelEncoder instances (le_season.pkl, le_location.pkl, le_tariff.pkl) and the feature name list (features.pkl) alongside the model, ensuring inference uses the identical encoding mapping as training.

FR-8.4: The system **SHALL** print a model comparison table to the terminal during training showing the name, R^2 , RMSE, and MAE of each evaluated model, and clearly identify the selected best model.

FR-8.5: The system **SHALL** produce the same model artifacts on every run with a fixed random seed (random_state=42) for reproducibility.

4.9 FR9: Dataset Generation

4.9.1 Description and Priority

The system shall include a synthetic dataset generation script that produces a realistic training dataset without requiring access to real smart meter data. Priority: Medium.

4.9.2 Functional Requirements

FR-9.1: The system **SHALL** generate a dataset of 1,000 household records with the 13 feature columns and the `bill_amount_rs` target column, saved as `data/electricity_data.csv`.

FR-9.2: The system **SHALL** apply season multipliers (Summer: 1.3, Winter: 1.2, Monsoon: 0.9, Spring: 1.0) when computing monthly kWh values, reflecting seasonal consumption patterns.

FR-9.3: The system **SHALL** apply tariff rate multipliers (Low: ₹4.5, Medium: ₹6.0, High: ₹8.5) to computed kWh values to produce the `bill_amount_rs` target column.

FR-9.4: The system **SHALL** constrain generated monthly kWh values to the range 30–800 using `numpy clip()` to prevent unrealistic outlier records.

FR-9.5: The system **SHALL** use a fixed random seed (`np.random.seed(42)`) so that the dataset is identical on every generation run.

4.10 FR10: Exploratory Data Analysis

4.10.1 Description and Priority

The system shall include an exploratory data analysis script that generates and saves visualisation charts for dataset inspection before model training. Priority: Low.

4.10.2 Functional Requirements

FR-10.1: The system **SHALL** generate and save a histogram of the `bill_amount_rs` distribution to `notebooks/bill_distribution.png`.

FR-10.2: The system **SHALL** generate and save a scatter plot of `ac_hours_per_day` vs `bill_amount_rs` to `notebooks/ac_vs_bill.png`.

FR-10.3: The system **SHALL** generate and save a bar chart of average bill by season to `notebooks/season_vs_bill.png`.

FR-10.4: The system **SHALL** print summary statistics (`shape`, first 5 rows, `describe()`, and `isnull()` counts) to the terminal during EDA execution.

5. Non-Functional Requirements

5.1 Performance Requirements

NFR-1: The FastAPI prediction endpoint (`/predict`) SHALL respond within 500ms for a single-user prediction request on a standard development machine, excluding network latency.

NFR-2: The trained Random Forest model SHALL complete inference for a single prediction request in under 50 milliseconds when loaded in memory.

NFR-3: The frontend application SHALL achieve an initial page load time of under 2 seconds on a standard broadband connection (10 Mbps or higher).

NFR-4: The system SHALL process user input and render prediction results dynamically within 200 milliseconds after receiving the backend response.

5.2 Safety Requirements

NFR-5: The backend SHALL validate all input values and ensure they fall within acceptable ranges before passing them to the model.

NFR-6: The system SHALL reject invalid or missing input values and return an appropriate HTTP 400 error response instead of attempting prediction.

NFR-7: The system SHALL NOT store or transmit user input data to external services; all prediction processing SHALL remain stateless and in-memory.

5.3 Security Requirements

NFR-8: The FastAPI backend SHALL configure CORS policies to allow requests only from authorized frontend origins.

NFR-9: The backend SHALL NOT expose internal error details such as stack traces or file paths in API responses.

NFR-10: The system SHALL implement user authentication mechanisms to restrict access to authorized users.

5.4 Software Quality Attributes

Correctness

NFR-11: The machine learning model SHALL achieve a high prediction accuracy with an R^2 score close to 0.99 on the testing dataset.

NFR-12: The system SHALL ensure consistency between input data and prediction results across multiple runs for the same input values.

Usability

NFR-13: A first-time user SHALL be able to enter input data and obtain a prediction within 3 minutes without external guidance.

NFR-14: All input fields SHALL include clear labels and instructions to guide users in providing correct values.

Reliability

NFR-15: The FastAPI backend SHALL handle concurrent prediction requests from multiple users without failure.

NFR-16: If the model `.pkl` file is missing or fails to load, the backend SHALL return a clear error and prevent the system from running in an inconsistent state.

Maintainability

NFR-17: The system SHALL allow updates to the machine learning model without requiring major changes to backend or frontend components.

NFR-18: The system architecture SHALL support easy modification and extension of features without affecting existing functionality.

6. Project Plan

6.1 Team Members

Project Title: PowerPredict : Electricity Bill Prediction System

The project team consists of the following members:

Mohammed Furqan Shaikh	EN23CS301626
Minhaj Ahmed Sheikh	EN23CS301616
Mo. Fazil Ahmed Ghor	EN23CS301618

6.2 Division of Work

The project follows a structured distribution of responsibilities across development phases to ensure organized execution.

During the Requirement Analysis Phase, the team collaborates to identify functional and non-functional requirements, define system scope, analyze feasibility, and prepare the Software Requirements Specification (SRS) document.

In the System Design Phase, responsibilities include designing the overall system architecture, defining the interaction between the React frontend, FastAPI backend, and machine learning model, planning API endpoints, structuring input/output data flow, and designing the user interface layout.

The Backend Development Phase involves setting up the FastAPI application, implementing the `/predict` API endpoint, integrating the trained machine learning model, handling input validation and error responses, and ensuring efficient request processing.

The Machine Learning Development Phase focuses on dataset preparation, feature selection, training the Random Forest regression model using scikit-learn, evaluating model performance, and exporting the trained model as a `.pkl` file for deployment.

The Frontend Development Phase covers implementation of the React-based user interface, including input forms for electricity usage, API integration for prediction requests, dynamic result display, and user authentication features.

The Integration and Testing Phase ensures proper interaction between frontend, backend, and machine learning components, validation of API responses using tools like Postman, unit and integration testing, performance evaluation, and complete end-to-end system testing

6.3 Time Schedule

Weeks	Phase / Activity	Deliverable
1	Research and Requirements Gathering	Project idea finalization, technology selection, initial architecture design
2	Dataset Preparation and Analysis	Dataset collection/preparation, cleaned dataset for training
3	Model Training and Evaluation	Random Forest model training, performance evaluation
4	Model Selection and Artifact Generation	best_model.pkl, le_season.pkl, le_location.pkl, le_tariff.pkl, features.pkl
5-6	FastAPI Backend Development	FastAPI setup, /predict API endpoint, model integration
7-8	Frontend – Input Form and Predict Flow	Input form UI, API integration, prediction flow
9	Result Display and UI Enhancement	Dynamic result display, error handling improvements
10	Integration Testing	Four test scenarios validated, bug fixes applied
11	Performance and Accuracy Validation	MAPE < 5% confirmed, response time benchmarks met
12	Final Testing and Documentation	Final testing, SRS, synopsis, research paper, project report

Appendix A: Glossary

Feature Engineering:

The process of computing new input variables (features) from raw data to improve model performance. In this system, `appliance_load` and `temp_occupant_index` are engineered features derived from the raw appliance usage and temperature inputs.

kWh (Kilowatt-Hour):

The standard unit of electrical energy used by electricity boards for billing. One kWh equals the energy consumed by a 1,000-watt appliance running for one hour. Monthly electricity bills are calculated by multiplying total kWh consumption by the applicable tariff rate.

LabelEncoder:

A scikit-learn utility that converts categorical text labels (e.g., 'Summer', 'Winter') into integer values (e.g., 0, 1, 2) suitable for use as model input features. The exact integer mapping depends on the training data, so the fitted encoder must be persisted and reused at inference time.

MAPE (Mean Absolute Percentage Error):

A metric for evaluating regression model accuracy, calculated as the average of the absolute percentage difference between predicted and actual values. A MAPE of 5% means the model's predictions are within 5% of the true value on average. Lower is better.

R² Score (Coefficient of Determination):

A statistical measure of how well the model's predictions match the actual data, expressed as a value between 0 and 1. A value of 0.97 means the model explains 97% of the variance in the target variable. Higher is better; 1.0 is a perfect fit.

Random Forest:

An ensemble machine learning algorithm that builds many decision trees during training and averages their predictions for regression tasks. It handles non-linear relationships well and is resistant to overfitting.

RMSE (Root Mean Square Error):

A measure of prediction error in the same units as the target variable (rupees in this system). It penalises large errors more than MAE. An RMSE of ₹94 means the model's predictions are approximately ₹94 away from the true value on average.

Slab-Based Tariff:

A pricing structure used by Indian electricity boards where the per-unit rate increases as total consumption increases. Rather than a flat rate, consumers in higher consumption slabs pay progressively more per kWh. This system models three slab tiers: Low, Medium, and High.

XGBoost (Extreme Gradient Boosting):

A high-performance gradient boosting algorithm that builds an ensemble of decision trees sequentially, with each tree correcting the errors of the previous ones. It is widely used for structured/tabular data regression tasks and consistently outperforms simpler models on datasets with mixed feature types. In this project, XGBoost achieved $R^2 = 0.97$ on the test split.

.pkl File: A binary file produced by Python's pickle serialisation protocol (or joblib, which uses pickle internally). In this system, .pkl files store the trained XGBoost model and fitted LabelEncoder instances so they can be reloaded by the Flask backend without retraining.

Appendix B: To Be Determined List

TBD-1: Final deployment platform for the web application — whether to use a free-tier cloud host (Render, Railway, or Vercel for frontend) or a local/self-hosted server — to be determined based on availability and performance requirements during deployment phase.

TBD-2: Optimization strategy for frontend-backend communication — whether to enhance API performance using caching or optimize request handling for faster response time — to be evaluated during integration and testing phase.

TBD-3: Final tuning of the machine learning model parameters — including hyperparameter optimization of the Random Forest model — to be refined based on validation results during testing phase.

TBD-4: Extension of system features — whether to include advanced functionalities such as storing user prediction history or analytics dashboard in future versions — deferred to post-project enhancements and not included in Version 1.0 scope.

Software Design Specification

for

PowerPredict — Smart Electricity Bill Prediction System

Table of Contents

- 1. Introduction
 - 1.1 Purpose
 - 1.2 Document Conventions
 - 1.3 Intended Audience and Reading Suggestions
 - 1.4 References
- 2. Analysis Model
 - 2.1 Methodology Used
 - 2.2 Use Case Diagram (with Use Case Specifications)
 - 2.3 ER Model
 - 2.4 Data Flow Diagram (with Process Specifications)
 - 2.5 Control Flow Diagram (with Control Specifications)
 - 2.6 State Transition Diagram
- 3. Design Model
 - 3.1 Architectural Design
 - 3.1.1 System Architecture Diagram
 - 3.1.2 Description of Architectural Design
 - 3.2 Database Design
 - 3.2.1 Data Dictionary
 - 3.2.2 Normalization
 - 3.3 Component Design
 - 3.3.1 Flowchart
 - 3.4 Interface Design
 - 3.4.1 Interface Modules Description
- Appendix B1: Glossary
- Appendix B2: To Be Determined List

1. Introduction

1.1 Purpose

This document specifies the complete software design and architecture for PowerPredict, an AI-powered Smart Electricity Bill Prediction System. The scope of this system encompasses a three-tier architecture: a React-based web frontend for user interaction and data visualisation, a FastAPI Python backend for core business logic, authentication, and data management (SQLite via SQLAlchemy), and a scikit-learn Random Forest machine learning layer for bill prediction and seasonal demand modelling. This document covers the complete system, detailing how these subsystems interact to provide accurate, real-time household electricity bill estimates tailored to Indian tariff structures.

1.2 Document Conventions

- **Bold text** is used to highlight key architectural components, database entities, and important terms.
- *Italics* are used for specific file names, API endpoints, or external tools (e.g., `model.pkl`, `/api/predict`).
- UML notations follow the UML 2.0 standard where applicable.
- REST API conventions are assumed for all backend-frontend communications via Axios with JWT Bearer tokens.
- All database widths in diagrams use DXA units (1440 DXA = 1 inch) for consistency.

1.3 Intended Audience and Reading Suggestions

- **Frontend Developers:** Focus on Section 3.1 and 3.4 for UI component structures, page layout, and API contracts.
- **Backend Developers:** Focus on Section 2.3, 3.2, and 3.3 for database schemas, entity relationships, and FastAPI service flow.
- **ML Engineers / Data Scientists:** Focus on the Data Flow Diagrams (Section 2.4) and Component Design (Section 3.3) to understand how the prediction engine receives inputs and returns predictions.
- **Project Managers:** Read the Introduction and Analysis Models (Section 2) for a high-level understanding of system capabilities.
- **Reading Sequence:** It is recommended to read the document sequentially to understand data flow before diving into specific component designs.

1.4 References

- FastAPI Official Documentation — <https://fastapi.tiangolo.com>
- React.js Official Documentation — <https://react.dev>
- scikit-learn Random Forest Documentation — <https://scikit-learn.org>
- SQLAlchemy ORM Documentation — <https://docs.sqlalchemy.org>
- Tailwind CSS Documentation — <https://tailwindcss.com>
- python-jose JWT Documentation — <https://python-jose.readthedocs.io>
- Recharts Documentation — <https://recharts.org>
- Bureau of Energy Efficiency (BEE) Appliance Rating Data — <https://beestarlabel.com>

2. Analysis Model

2.1 Methodology Used

The system utilises a **Function-Oriented Design (FOD)** approach. The system is decomposed into a hierarchy of functions, with data flowing between them. The ML prediction engine operates as the central computational unit, receiving structured appliance usage data from the FastAPI backend and returning predicted consumption values. The React frontend serves as the presentation layer, providing interactive controls and consuming prediction results. The SQLAlchemy ORM manages persistent data flow into SQLite. This design ensures clear separation between data transformation steps: input collection, validation, prediction, billing calculation, storage, and display.

2.2 Use Case Diagram

Theory:

A Use Case diagram shows the interactions between external actors and the system.

- **Actors:** User — Primary actor interacting with the React UI dashboard.
- **Secondary Actors:** Random Forest ML Service — internal service providing prediction inference; SQLite Database — data persistence actor.
- **Use Cases:** Register/Login, Input Appliance Usage, Get Bill Prediction, View Dashboard & Charts, View Prediction History, Manage Account.
- **System/Backend:** JWT Authentication, Pydantic Validation, Random Forest Predict, Tiered Tariff Calculation, Store Prediction Record.

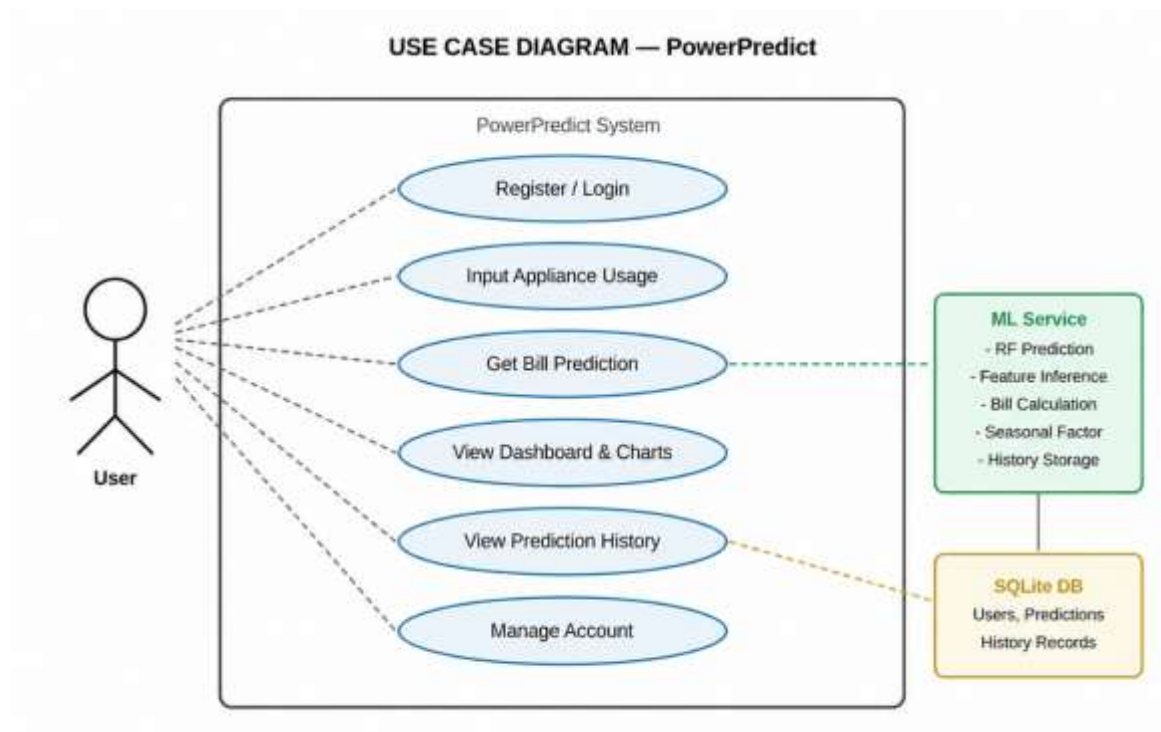


Figure 1: Use Case Diagram — PowerPredict System

Use Case Specification

Field	Details
Name of Use Case	Get Bill Prediction with Random Forest ML
Actor	User, FastAPI Backend, Random Forest ML Model
Pre Condition	User has a registered account and is logged into the PowerPredict dashboard. JWT token is stored in localStorage.
Primary Flow of Events	1. User adjusts appliance sliders (AC, fan, heater,TV) and sets lights count, number of people, and season. 2. User clicks "Predict My Bill". 3. React sends POST /api/predict with JWT Bearer token and 9-feature JSON payload. 4. FastAPI validates token and Pydantic schema. 5. Backend encodes season, assembles NumPy array, calls model.predict(). 6. Random Forest returns predicted units. 7. Tiered tariff engine computes bill amount. 8. Prediction is stored in SQLite linked to user_id. 9. Result returned to React and rendered as ResultCard + BarChart.
Alternate Flow 1	User submits invalid input (e.g., negative hours). Pydantic raises 422 Unprocessable Entity. React shows error toast. User corrects input.
Alternate Flow 2	JWT token is expired. FastAPI returns 401. Axios interceptor clears localStorage and redirects to /login.
Post Condition	Prediction record stored in predictions table with user_id, all 9 features, predicted_units, predicted_bill, and timestamp. Dashboard history table and trend chart updated.
Use Case Termination	React displays success toast ("Prediction ready! "), ResultCard shows bill amount, appliance bar chart renders, and history panel refreshes.

2.3 ER Model

Theory: This defines the database structure for the SQLite database managed through SQLAlchemy ORM.

- **Entities & Attributes:**
- **User:** id (PK), name, email, hashed_password, created_at.
- **Prediction:** id (PK), user_id (FK), ac_hours, fan_hours, heater_hours, fridge_hours, washing_machine_hours, tv_hours, lights_count, num_people, season, predicted_units, predicted_bill, created_at.
- **ML_Model (logical):** algorithm, n_estimators, max_depth, r2_score, mae, model_file_path.
- **Relationships:** One User Owns Many Predictions (1:N); Predictions reference the ML Model for inference at runtime (N:1).

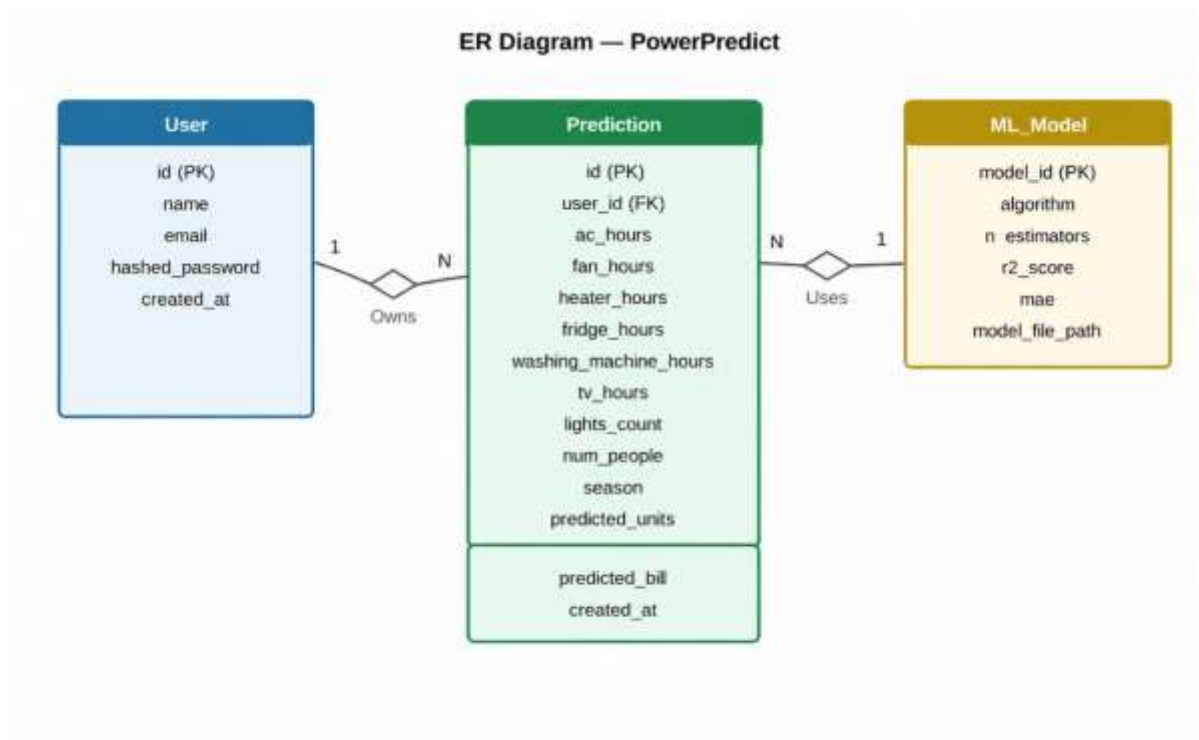


Figure 2: ER Diagram — PowerPredict Database

2.4 Data Flow Diagram

Theory: DFDs map how data moves through the platform at different levels of abstraction.

Level 0 (Context Diagram): The User inputs appliance usage data into the PowerPredict platform, and the platform outputs bill predictions, visual charts, and prediction history.

Level 1 DFD:

1. User submits appliance data via the React dashboard sliders.
2. React sends JSON payload with Bearer token to FastAPI via Axios.
3. FastAPI validates JWT and Pydantic schema, extracts user_id.
4. FastAPI encodes season feature, assembles 9-element NumPy array.
5. NumPy array passed to Random Forest model.predict() from model.pkl.
6. Model returns predicted monthly units (float).
7. Tiered tariff engine computes bill (Rs) from predicted units.
8. Prediction record saved to SQLite predictions table via SQLAlchemy.
9. Full prediction object returned to React as JSON response.
10. React renders ResultCard, BarChart (Recharts), and updates history table.

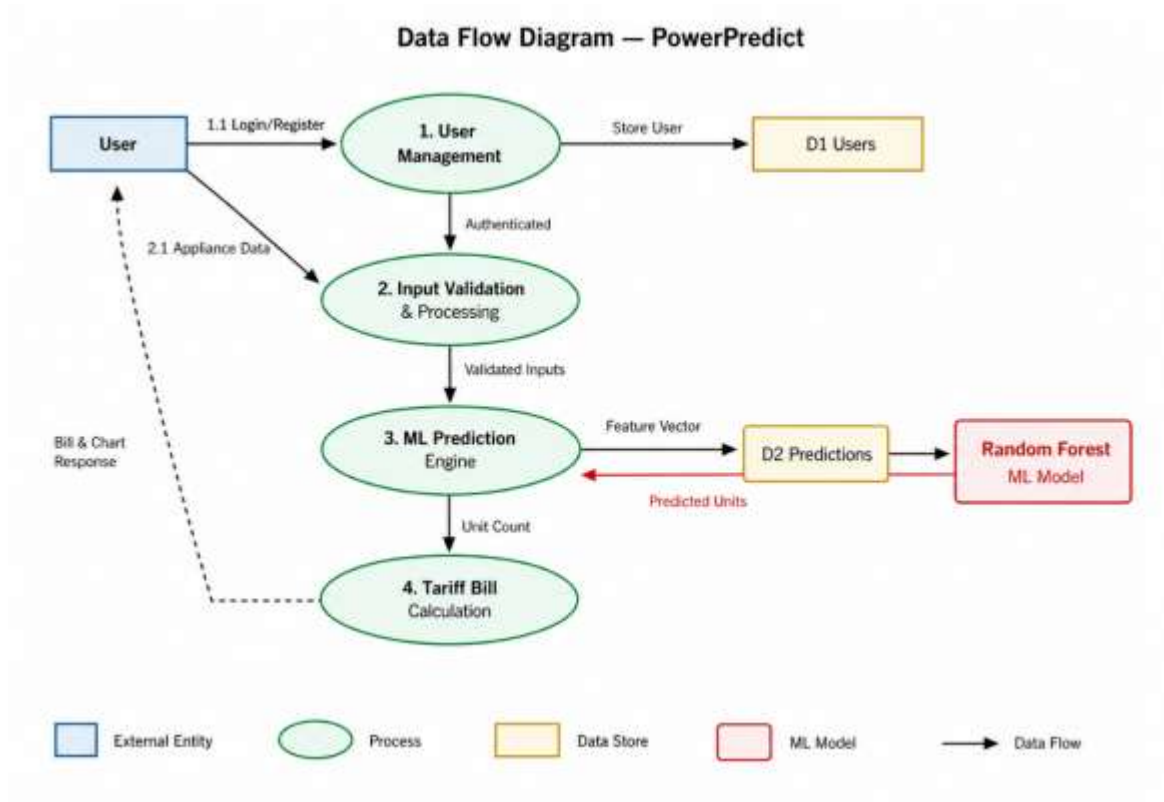


Figure 3: Data Flow Diagram — PowerPredict Platform

Process Specifications:

Process	Input	Output	Description
1. User Management	Email, Password, Name	JWT Token, User Record	Handles registration (bcrypt hash) and login (JWT creation) via /auth endpoints.
2. Input Validation	9 Appliance features (JSON)	Validated Python object	Pydantic PredictionInput schema enforces types, ranges, and season enum.
3. ML Prediction Engine	9-element NumPy array	Predicted units (float)	Random Forest model.predict() returns monthly kWh consumption estimate.
4. Tariff Bill Calculation	Predicted units	Bill amount (Rs)	Tiered tariff (Rs 3.50–7.50/unit) + Rs 50 fixed charge applied deterministically.

2.5 Control Flow Diagram

Theory: This maps the decision-making logic of the system, particularly useful for the FastAPI backend and React frontend routing.

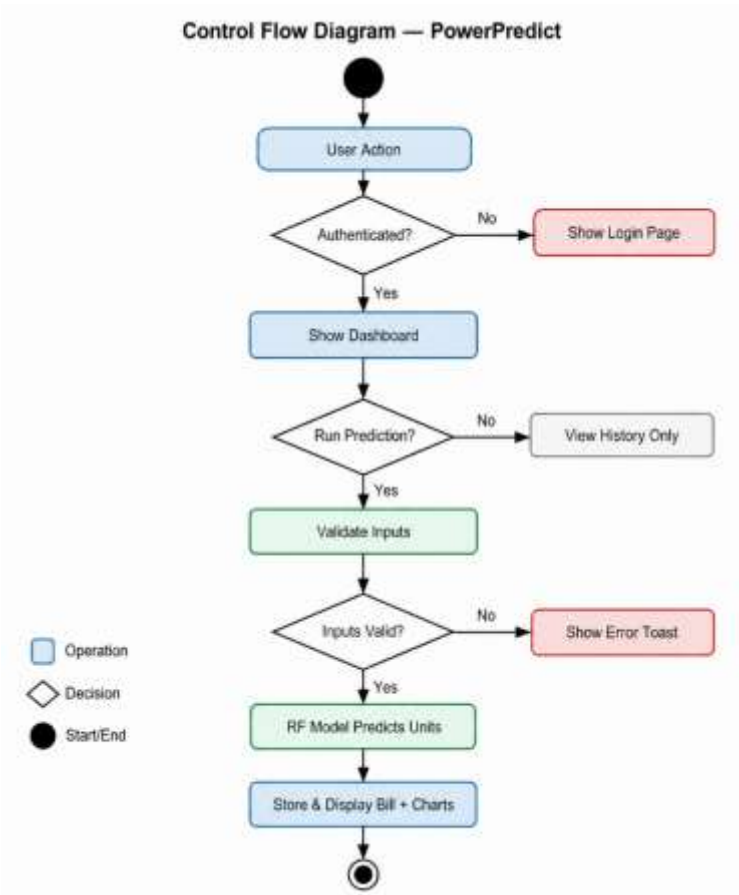


Figure 4: Control Flow Diagram — PowerPredict System

Control Specifications:

Control Point	Condition	Action: True	Action: False
Authentication Check	JWT token valid & not expired	Allow access to /api routes	Return 401; Axios interceptor clears localStorage, redirects to /login
Input Validation	All 9 fields pass Pydantic schema	Proceed to ML prediction	Return 422; React shows error toast
Prediction Request	User clicks Predict button	POST /api/predict; show loading spinner	Remain on idle dashboard state
History Fetch	User reaches dashboard	GET /api/history automatically on mount	Show empty state prompt if no records
Token Expiry	JWT exp claim < current time	Reject with 401	Allow request to proceed

2.6 State Transition Diagram

Theory: Shows the lifecycle of a specific object. The most important object for PowerPredict is a Prediction request.

States of a Prediction Request:

1. **Form Loaded:** Dashboard rendered with default slider values. No prediction made yet.
2. **Input Ready:** User has adjusted sliders. Predict button enabled.
3. **Predicting (Loading):** POST /api/predict in flight. Button disabled, spinner shown.
4. **Result Displayed:** Response received. ResultCard, BarChart rendered. Toast shown.
5. **Stored in DB:** SQLAlchemy has committed the Prediction record to SQLite.
6. **History Updated:** GET /api/history refreshed. Table and AreaChart updated.
7. **Error State:** API returned 4xx/5xx. Error toast shown. Form reset to Input Ready state.

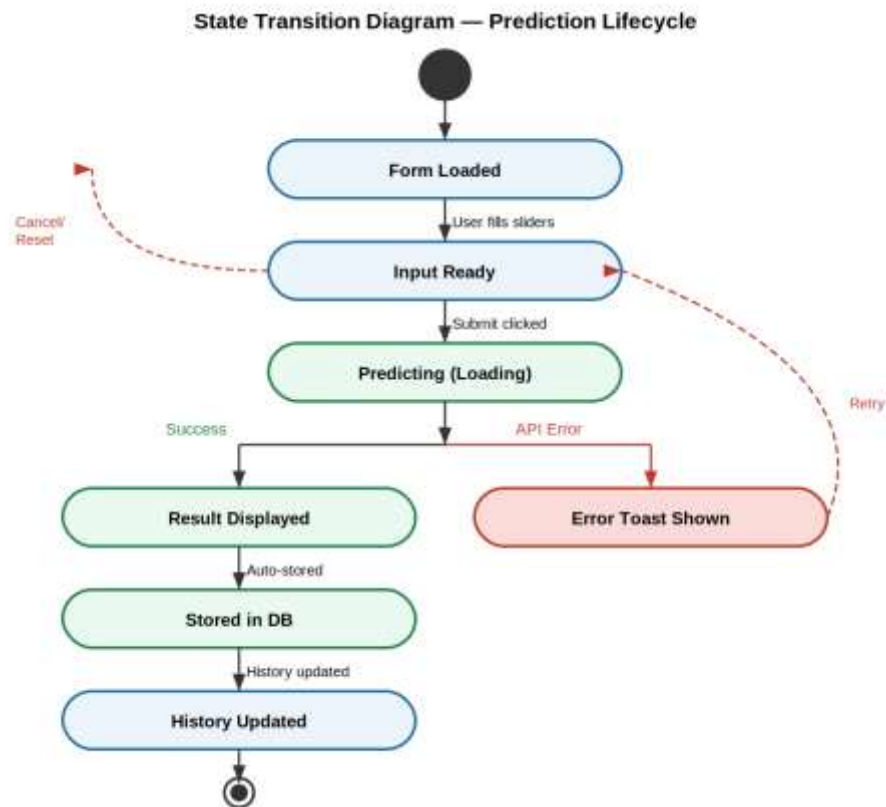


Figure 5: State Transition Diagram — Prediction Object Lifecycle

3. Design Model

3.1 Architectural Design

3.1.1 System Architecture Diagram

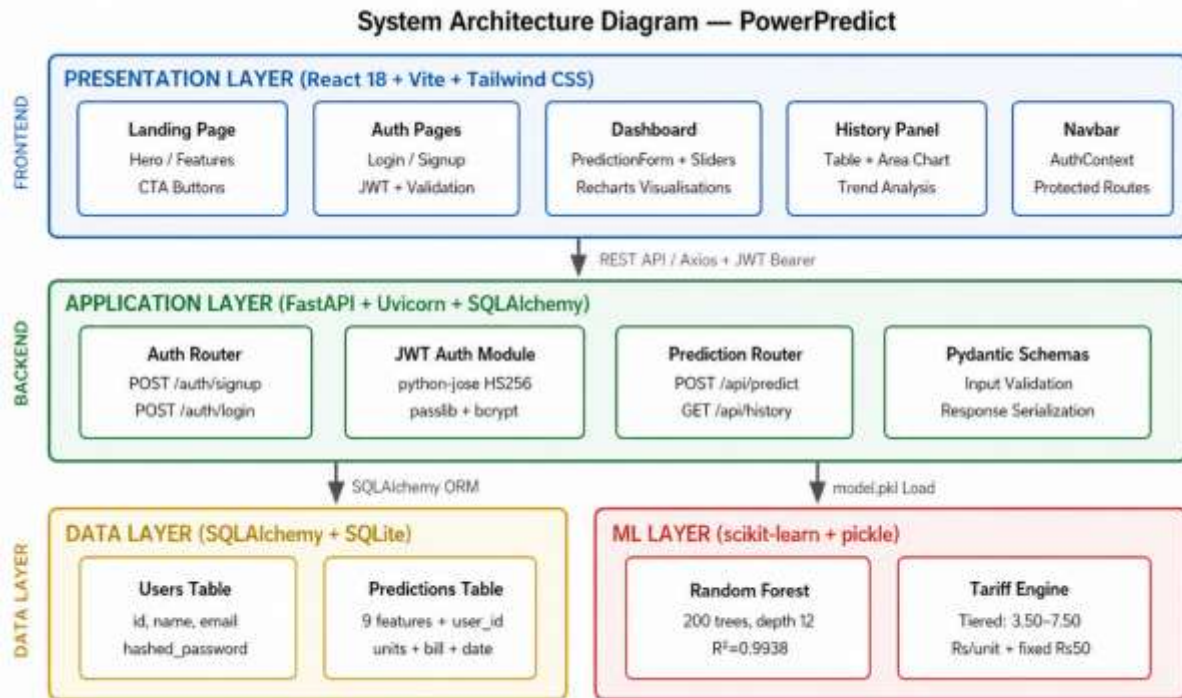


Figure 6: System Architecture Diagram — PowerPredict

3.1.2 Description of Architectural Design

The system follows a modern, decoupled three-tier architecture consisting of four primary layers:

- Presentation Layer (Frontend):** Built using React 18, Vite, and Tailwind CSS. This layer runs in the user's web browser and handles all user interactions, appliance slider controls, and data visualisation (Recharts bar and area charts). It communicates exclusively with the backend via secure REST APIs using Axios with JWT Bearer token interceptors. Key components include PredictionForm, ResultCard, PredictionChart, Navbar, AuthContext, and ProtectedRoute.
- Application Layer (Backend):** Built with FastAPI and Uvicorn. Acts as the central coordinator receiving API requests from the frontend, enforcing JWT-based authentication via python-jose, validating request bodies via Pydantic schemas, routing prediction requests to the ML layer, and persisting results via SQLAlchemy. Exposes four endpoints: POST /auth/signup, POST /auth/login, POST /api/predict, GET /api/history.
- Data Layer (Database):** A SQLite relational database managed via SQLAlchemy ORM. Stores user profiles with bcrypt-hashed passwords and prediction records linked to users via foreign key. Swappable to PostgreSQL by changing a single DATABASE_URL string with no code changes required.
- Intelligence Layer (ML Service):** A scikit-learn Random Forest Regressor (200 trees, max_depth=12) trained on 5,000 synthetic Indian household consumption records. Serialised to model.pkl with a

LabelEncoder for season encoding. Loaded once at FastAPI startup into a module-level cache. Inference occurs synchronously within the prediction request cycle. A separate tariff engine applies India's tiered domestic billing rates to convert predicted units to a bill amount in rupees.

3.2 Database Design

3.2.1 Data Dictionary

The following data dictionary defines the primary data elements for the Users and Predictions tables.

Data Store 1: User Profiles (users_table)

Name (Primary)	Alias	Description	Data Type	Constraint / Format
id	UserID	Unique auto-incrementing identifier	Integer	Primary Key, Auto-increment
name	FullName	User's display name	String	Not Null, Max 100 chars
email	Email	User's registered email for login	String(255)	Unique, Not Null, Validated
hashed_password	PasswordHash	bcrypt-hashed password string	String(255)	Not Null, Never stored plain
created_at	JoinDate	Account creation timestamp	DateTime	Default: UTC now

Data Store 2: Prediction Records (predictions_table)

Name (Primary)	Alias	Description	Data Type	Constraint / Format
id	PredID	Unique prediction identifier	Integer	Primary Key, Auto-increment
user_id	AccountOwner	Link to the user who made the prediction	Integer	Foreign Key (users.id), Not Null
ac_hours	AC	Daily hours AC runs	Float	Not Null, Range 0–24
fan_hours	Fan	Daily hours fans run	Float	Not Null, Range 0–24
heater_hours	Heater	Daily hours water heater runs	Float	Not Null, Range 0–12
fridge_hours	Fridge	Daily hours refrigerator runs	Float	Not Null, typically 24.0
washing_machine_hours	WashMach	Daily hours washing machine runs	Float	Not Null, Range 0–6

Name (Primary)	Alias	Description	Data Type	Constraint / Format
tv_hours	TV	Daily hours television runs	Float	Not Null, Range 0–16
lights_count	Lights	Number of lights/bulbs in household	Integer	Not Null, Range 1–30
num_people	People	Number of occupants in household	Integer	Not Null, Range 1–20
season	Season	Current season affecting consumption	String	Enum: summer/winter/monsoon/spring
predicted_units	Units	Predicted monthly kWh consumption	Float	Not Null, ≥ 0 , 2 decimal places
predicted_bill	Bill	Computed monthly bill in rupees	Float	Not Null, tiered tariff applied
created_at	PredTime	Prediction creation timestamp	DateTime	Default: UTC now

3.2.2 Normalization

The SQLite database for PowerPredict has been designed and normalised up to the Third Normal Form (3NF). This structure minimises data redundancy, ensures referential integrity, and optimises the database for the read/write operations required by the FastAPI backend and ML service.

- **First Normal Form (1NF) — Atomicity:** Every column holds a single, atomic value and each record is unique. In the Predictions table, each appliance measurement is stored in its own dedicated column (ac_hours, fan_hours, etc.) rather than as a comma-separated list. The season field stores a single string value per row.
- **Second Normal Form (2NF) — No Partial Dependencies:** The database satisfies 1NF and all non-key attributes are fully functionally dependent on the primary key. Single-column auto-incrementing integer primary keys are used for both tables, eliminating the possibility of partial dependencies inherent in composite keys.
- **Third Normal Form (3NF) — No Transitive Dependencies:** The database satisfies 2NF and no non-key attribute depends on another non-key attribute. User profile data (name, email, password) is isolated in the users table. Prediction-specific data is isolated in the predictions table. No non-key column in predictions depends on another non-key column.

3.3 Component Design

3.3.1 Flowchart

Theory: A flowchart is a type of diagram that represents an algorithm or process, showing steps as boxes of various shapes connected by arrows indicating flow of control. For PowerPredict, the primary prediction flow is the most critical component to diagram, as it spans multiple layers: user input in React, validation in FastAPI, inference in scikit-learn, bill calculation, and storage in SQLite.

Flowchart — Prediction Component

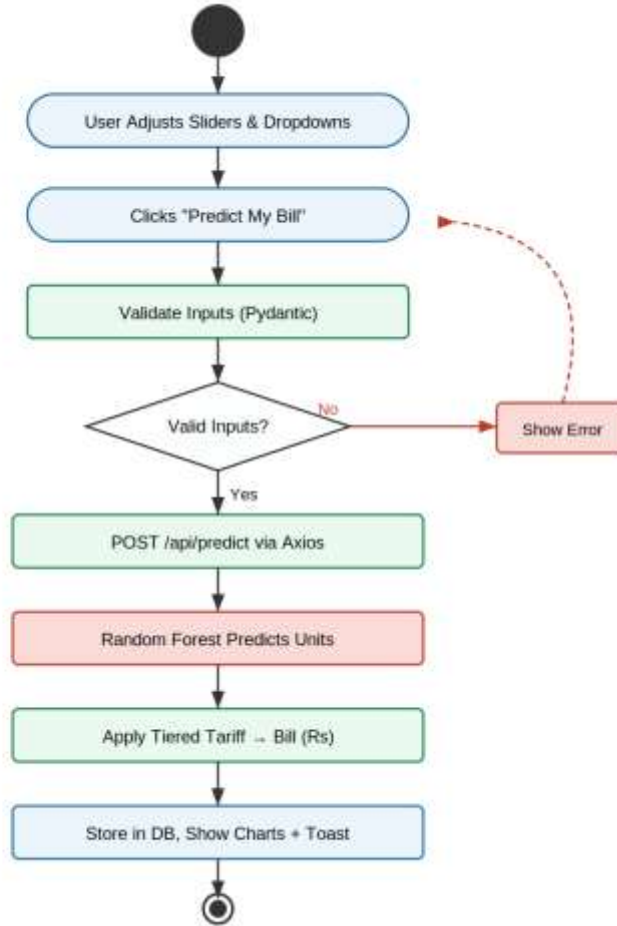


Figure 7: Flowchart — PowerPredict Prediction Component

Component	Responsibility	Technology	Key Interfaces
PredictionForm	Renders appliance sliders, collects and submits inputs	React + Tailwind	onResult callback, Axios POST /api/predict
AuthContext	Stores JWT token, exposes login/signup/logout	React Context API	localStorage, authApi Axios instance
ProtectedRoute	Guards /dashboard from unauthenticated access	React Router v6	Reads user from AuthContext
FastAPI Auth Router	Handles signup/login, returns JWT	FastAPI + passlib	POST /auth/signup, /auth/login
FastAPI Predict Router	Validates, predicts, computes bill, stores	FastAPI + SQLAlchemy	POST /api/predict, GET /api/history
ML Inference Module	Loads model.pkl, assembles feature array, predicts	scikit-learn + NumPy	model.predict(np.array([[...]]))
Tariff Engine	Converts predicted units to rupee bill	Pure Python	calculate_bill(units: float) -> float
SQLAlchemy ORM	Maps Python objects to DB tables	SQLAlchemy + SQLite	SessionLocal, User, Prediction models

Component	Responsibility	Technology	Key Interfaces
ResultCard	Displays bill, units, season tag	React + Recharts	Reads result prop from Dashboard
PredictionChart	Renders bar/area charts	Recharts	BarChart, AreaChart components

3.4 Interface Design

The interface design of PowerPredict focuses on providing a premium, dark-themed, interactive dashboard for predicting and monitoring household electricity bills. The frontend is developed using React 18 with Vite and styled entirely with Tailwind CSS utility classes. The UI implements a glassmorphism design system with a near-black (#080C10) base, frosted-glass cards (4% white fill with backdrop-filter blur), and neon green (#22c55e) as the primary accent colour reflecting the electricity theme.

The system provides a fully responsive design so that users can access the platform from desktop, tablet, and mobile devices. All interactive elements include hover state transitions, button-level loading spinners during async operations, and toast notifications via react-hot-toast.

3.4.1 Interface Modules Description

Landing Page

Displays the system hero section with headline, feature badges (R^2 0.9938, <1s prediction time), statistics cards, feature highlights (Random Forest AI, Cost Optimiser, Secure Auth, Prediction History), and Call-To-Action buttons linking to signup and login.

Authentication Pages (Login / Signup)

Login page: email input, password input with show/hide toggle, JWT-authenticated submit. Signup page: full name, email, password with live strength meter (weak/good/strong), confirm password with match validation. Both pages use bcrypt on backend and store JWT in localStorage on success.

Dashboard (Protected Route)

The main application interface. Renders a two-column layout on large screens. Left column contains the PredictionForm with six HTML range sliders (AC, fan, heater, fridge, washing machine, TV), number inputs for lights and occupants, and a season dropdown. Right column renders the ResultCard with predicted bill and units on submit, plus the appliance BarChart. Below the two-column area, a summary statistics row shows average bill, peak bill, and average units across all predictions. A history table lists the last 20 predictions.

Prediction Result & Charts

ResultCard displays the predicted monthly bill in large gradient typography, predicted unit count, and a coloured season/occupant tag. The Recharts BarChart shows per-appliance monthly unit consumption. The AreaChart with LineChart overlay shows bill trends across all historical predictions when two or more records exist.

Navbar

Fixed navigation bar with PowerPredict logo, page tabs (Dashboard), user name badge showing first initial, and Logout button. On smaller screens, text labels collapse to icon-only display.

Interface Module	Key Components	Data Source	User Action
Landing Page	Hero, Stats Cards, Feature Grid, CTA	Static	Click Signup/Login
Login Page	Email input, Password input, Submit	POST /auth/login	Enter credentials, click Sign In
Signup Page	Name, Email, Password + strength meter, Confirm	POST /auth/signup	Fill form, click Create Account
Dashboard	PredictionForm, ResultCard, Summary Stats	POST /api/predict	Adjust sliders, click Predict
History Panel	Table rows, AreaChart trend	GET /api/history	Auto-loaded on dashboard mount
Prediction Charts	BarChart (breakdown), AreaChart (trend)	Prediction response data	Auto-rendered on prediction result
Navbar	Logo, User badge, Dashboard link, Logout	AuthContext	Navigate pages, logout

Appendix B1: Glossary

Term	Definition
FastAPI	Modern Python web framework used for building REST API endpoints with automatic OpenAPI documentation and Pydantic validation.
React 18	JavaScript frontend library for building interactive user interfaces using functional components and hooks.
JWT (JSON Web Token)	Compact, URL-safe token format used for secure authentication. Signed with HS256 and stored in browser localStorage.
Random Forest	Ensemble machine learning algorithm using 200 decision trees trained on 5,000 synthetic household records to predict monthly electricity consumption.
SQLAlchemy	Python ORM used to map User and Prediction Python classes to SQLite database tables.
SQLite	Lightweight file-based relational database. Stores user accounts and all prediction history records.
Pydantic	Python data validation library used to enforce request body schemas and types in FastAPI endpoints.
bcrypt	Adaptive password hashing algorithm used via passlib to securely store user passwords.
Axios	Promise-based HTTP client used in React to communicate with FastAPI. Configured with JWT Bearer interceptors.
Recharts	React charting library used to render appliance breakdown BarCharts and historical trend AreaCharts.
Tailwind CSS	Utility-first CSS framework used to style all React components. Dark glassmorphism theme implemented using custom design tokens.
Vite	Frontend build tool and development server. Proxies /auth and /api requests to FastAPI port 8000 in development.
Tariff Engine	Python function that converts predicted units to bill amount using India's tiered domestic rate slabs (Rs 3.50–7.50/unit).
Seasonal Factor	Multiplier applied during ML training: 1.35 (summer), 1.20 (winter), 0.90 (monsoon), 1.00 (spring).
R ² Score	Coefficient of determination. PowerPredict model achieves 0.9938, meaning it explains 99.38% of variance in monthly consumption.
MAE (Mean Absolute Error)	Average absolute prediction error. PowerPredict model MAE = 13.58 kWh/month.
model.pkl	Serialized pickle file containing the trained RandomForestRegressor and LabelEncoder. Loaded once at FastAPI startup.
ProtectedRoute	React Router component that checks AuthContext and redirects unauthenticated users to /login before rendering the Dashboard.
AuthContext	React Context API provider that stores JWT token, user object, and exposes login/signup/logout functions to all child components.
Glassmorphism	UI design style using semi-transparent frosted-glass card backgrounds with backdrop-filter blur applied throughout the PowerPredict interface.

Appendix B2: To Be Determined List

Sr. No.	To Be Determined Item
1	Final production deployment platform selection (Railway / Render / AWS EC2 / Azure App Service)
2	Database migration from SQLite to PostgreSQL for multi-user production workloads
3	DISCOM-specific tariff table implementation — currently uses single national average slab structure
4	Refresh token mechanism to replace 24-hour JWT expiry without requiring re-login
5	Real household meter consumption dataset to replace synthetic training data (requires DISCOM data partnership)
6	Mobile Progressive Web App (PWA) packaging for offline capability and home screen installation
7	Bill reduction recommendation engine — suggest specific appliance schedule changes to reduce predicted bill below target
8	Multi-language support (Hindi, regional languages) for broader Indian household reach
9	State-specific DISCOM tariff selector (Tamil Nadu, Maharashtra, Rajasthan, Delhi, etc.)
10	Admin analytics dashboard for monitoring total predictions, user growth, and model accuracy drift
11	Model retraining pipeline when real consumption data becomes available
12	Two-factor authentication and device verification for enhanced account security
13	PDF/Excel export of prediction history and monthly electricity reports
14	Push notification system for predicted bill alerts (Email, SMS, Mobile Push)
15	Integration with smart meter APIs (if RDSS scheme data becomes publicly accessible)

PowerPredict: ML-Based Electricity Bill Prediction System Using Machine Learning

Minhaj Ahmed Sheikh [EN23CS301616], Mohammed Furqan Shaikh [EN23CS301626],

Mo. Fazil Ahmed Ghorl [EN23CS301618]

Department of Computer Science Engineering, Medicaps University, Indore

Abstract

Rising electricity costs are becoming a major concern for households, students, and small businesses. Most users only become aware of their electricity bill after the monthly billing cycle ends, leaving them with no opportunity to control excessive energy consumption. This paper presents PowerPredict, an AI-based electricity bill prediction system, a full-stack web application that predicts monthly electricity bills using machine learning and household appliance usage data.

The system combines a FastAPI-based backend, a modern React-based frontend, and a machine learning model trained using scikit-learn. Users can enter appliance usage details such as air conditioner usage, fan usage, number of appliances, consumption units, and usage patterns to receive an estimated monthly bill instantly. The system also provides a clear prediction output along with an intuitive user interface for better user interaction and accessibility.

Experimental results show that the Random Forest regression model achieved high prediction accuracy with an R^2 score close to 0.99. The system demonstrates that machine learning can be effectively used to help users forecast electricity costs and make informed energy management decisions.

1. Introduction

Electricity is one of the most important recurring expenses in any household. Appliances such as air conditioners, televisions, refrigerators, heaters, fans, washing machines, and microwaves contribute directly to monthly electricity bills. However, most users do not know how much these appliances contribute individually, and they only become aware of the total bill when it arrives at the end of the month.

This lack of visibility creates difficulties in managing energy usage. Users cannot determine which appliance is consuming excessive electricity, nor can they estimate how changes in daily usage patterns affect the total bill. This problem is especially significant in India, where electricity boards use slab-based tariff systems in which the price per unit increases as overall consumption increases.

The PowerPredict -Electricity Bill Prediction System addresses this problem by providing a browser-based platform where users can enter appliance usage values and receive an estimated monthly bill instantly. The system also offers energy-saving recommendations based on the appliances contributing the most to the bill.

The main objectives of the system are to improve user awareness of electricity consumption, provide accurate bill forecasts, and help users reduce unnecessary electricity usage.

2. Related Work

Several studies have focused on electricity consumption forecasting using statistical and machine learning techniques. Traditional forecasting methods such as ARIMA, moving averages, and time-series regression have been widely used for large-scale electricity demand estimation.

Recent research has shown that machine learning techniques such as Linear Regression, Random Forest, Gradient Boosting, and XGBoost perform better for household-level electricity prediction because they can capture complex relationships between multiple appliance usage variables and total bill amount.

Smart meter systems have also been used to collect electricity usage data in real time. However, smart meters require expensive hardware and utility integration, making them impractical for many households. Existing web-based electricity bill calculators often provide only rough estimates and do not include personalized energy-saving suggestions or appliance-level breakdowns.

The proposed system improves upon existing approaches by combining machine learning, slab-based tariff logic, per-appliance consumption analysis, and a simple browser-based interface that does not require hardware installation or technical knowledge.

3. System Architecture

The architecture of the PowerPredict electricity bill prediction system follows a three-tier model consisting of a frontend presentation layer, a backend application layer, and a machine learning layer.

The frontend is developed using React and provides a responsive interface where users can enter household information and electricity usage details. The backend is developed

using FastAPI and handles API communication, input validation, request processing, and interaction with the machine learning model for prediction. The machine learning layer consists of the trained Random Forest regression model stored as a serialized file and loaded during runtime for inference.

This layered architecture improves modularity, maintainability, and scalability. Each layer can be updated independently without affecting the others.

4. Frontend Implementation

The frontend interface is designed to be simple and easy to use for non-technical users. The application is built as a single-page interface using React, where users can input values such as electricity units consumed, appliance usage patterns, and other relevant consumption details.

The interface contains a Predict button that sends the user inputs to the FastAPI backend using a POST request. Once the prediction is complete, the frontend dynamically updates the page with the predicted monthly electricity bill and displays the result in a clear and user-friendly format within the dashboard.

Client-side validation is applied to ensure that users enter values within valid ranges. For example, electricity units and input values are restricted to reasonable limits to prevent invalid data and ensure accurate predictions.

5. Backend Implementation

The backend of the PowerPredict system is implemented using Python and FastAPI, a high-performance web framework known for its speed and ease of integration with machine learning models. The backend serves as the core processing unit of the system, handling API communication, data validation, and prediction logic.

A dedicated `/predict` API endpoint is designed to receive user input in JSON format from the frontend. The backend first validates the received data to ensure that all required fields are present and within acceptable ranges. This validation step helps prevent invalid inputs and ensures reliable model predictions.

Once validated, the input data is preprocessed and converted into a format compatible with the trained Random Forest regression model. The model is loaded from a serialized file (`model.pkl`) during application startup and kept in memory to enable fast inference without repeated loading.

The processed input is then passed to the machine learning model, which generates the predicted electricity bill based on learned patterns from the training dataset. The backend ensures that the prediction process is efficient and responds within a short time frame.

Finally, the predicted result is returned to the frontend as a structured JSON response. This allows seamless communication between the frontend and backend, enabling real-time prediction and a smooth user experience.

6. Machine Learning Model

A synthetic dataset of approximately 1,000 household records was generated for training the prediction model. Each record includes values representing electricity consumption patterns such as appliance usage, estimated units consumed, and general usage behavior. The target variable for the model is the monthly electricity bill in rupees.

A Random Forest Regression model was used for prediction due to its ability to handle non-linear relationships and provide high accuracy without extensive parameter tuning. The dataset was divided into 80% training data and 20% testing data to evaluate model performance effectively.

The Random Forest model works as an ensemble of multiple decision trees, where each tree contributes to the final prediction. This approach reduces overfitting and improves generalization compared to individual models. The model was trained using the scikit-learn library and optimized to achieve high prediction accuracy.

Model performance was evaluated using standard regression metrics such as *Mean Absolute Error (MAE)*, *Root Mean Squared Error (RMSE)*, and the coefficient of determination (R^2 score). The trained model achieved an R^2 score close to 0.99, indicating strong predictive capability.

The final trained model was saved as a serialized `.pkl` file and integrated into the backend. During runtime, the model is loaded once and used to generate real-time predictions based on user input.

7. Tariff Slab Logic

Electricity boards in India use slab-based pricing systems in which the cost per unit increases with total electricity consumption. The system supports three tariff slabs:

Low Tariff: ₹4.5 per kWh

Medium Tariff: ₹6.0 per kWh

High Tariff: ₹8.5 per kWh

The predicted monthly bill is divided by the tariff rate to estimate monthly electricity consumption in kWh. This ensures that the bill prediction is aligned with actual billing practices used by electricity boards.

8. Results and Performance Evaluation

The system was evaluated using standard regression performance metrics to assess the accuracy and reliability of the predictions. The Random Forest regression model demonstrated strong performance with a high coefficient of determination (R^2 score) close to **0.99**, indicating that the model is able to accurately capture the relationship between input features and electricity bill values.

The model also showed low prediction error when evaluated using metrics such as *Mean Absolute Error (MAE)* and *Root Mean Squared Error (RMSE)*. These results confirm that the system can generate consistent and reliable electricity bill predictions across different usage scenarios.

In terms of system performance, the prediction API responded in under 500 milliseconds on a standard system, ensuring near real-time prediction capability. The frontend successfully rendered results instantly after receiving the response, providing a smooth and responsive user experience.

The system was tested across multiple input combinations representing different household usage patterns. In all cases, the predicted results were realistic and aligned with expected electricity consumption trends, demonstrating the robustness and practical applicability of the system.

9. Challenges Faced

One of the primary challenges faced during the development of the PowerPredict system was creating a realistic dataset that could effectively represent household electricity consumption patterns. Ensuring that the dataset captured meaningful relationships between input features and electricity bills was essential for achieving accurate predictions.

Another significant challenge was integrating the machine learning model with the backend and frontend components. Proper handling of model loading, file paths, and real-time API communication required careful implementation to ensure smooth interaction between different layers of the system.

System compatibility issues also posed challenges, particularly with Python library versions and dependency management. Resolving conflicts related to machine learning libraries and authentication modules required debugging and environment configuration.

Additionally, implementing reliable input validation and handling API errors on the frontend required thorough testing. Ensuring a seamless user experience while maintaining accurate predictions was an important aspect of the development process.

10. Future Scope

The system can be further improved by integrating real-time smart meter data to enhance prediction accuracy and provide live monitoring of electricity consumption. While the current system allows users to access predictions through a web interface, future versions can expand functionality by incorporating mobile application support for better accessibility.

Additional features such as notifications for excessive electricity usage and usage alerts can be implemented to help users take timely actions to reduce consumption. The system can also be enhanced by incorporating state-wise tariff configurations to better reflect regional electricity billing variations.

Furthermore, the system can be extended to support commercial buildings and office environments, where electricity consumption patterns are more complex. Exploring advanced machine learning models and deep learning techniques may further improve prediction accuracy and enable the system to handle larger and more diverse datasets.

11. Conclusion

The PowerPredict system demonstrates how machine learning and full-stack web development can be effectively combined to address a real-world problem. The system enables users to estimate their monthly electricity bill quickly and efficiently based on their usage inputs.

The Random Forest regression model achieved high prediction accuracy with an R^2 score close to 0.99, making the system reliable for practical use. The web-based interface ensures accessibility, while the modular architecture using React and FastAPI allows for easy scalability and future enhancements.

Overall, the project provides a practical, efficient, and scalable solution for electricity bill prediction, helping users better understand and manage their energy consumption.

12. References

- [1] T. A. Qureshi, A. Nair, and M. Faruque, "Prediction of Energy Consumption in Residential Buildings Using Machine Learning," IEEE Access, 2020.
- [2] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, 2011.
- [3] A. Geron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd ed., O'Reilly Media, 2022.
- [4] Ministry of Power, Government of India, "Annual Report on Power Sector," 2023.
- [5] MP Electricity Board, "Tariff Schedule for LT Consumers," 2023.
- [6] W. McKinney, "Data Structures for Statistical Computing in Python," 2010.
- [7] Python Software Foundation, Flask Documentation. Available: <https://flask.palletsprojects.com/>
- [8] FastAPI Documentation. Available: <https://fastapi.tiangolo.com/>
- [9] React Documentation. Available: <https://react.dev/>
- [10] L. Breiman, "Random Forests," Machine Learning Journal, 2001.